MAJOR PROJECT REPORT

on

Transact Rite

- *Your master companion for all financial transactions and note handling.*

SUBMITTED TO –

DR. SANTOSH KUMAR YADAV

(LECTURER) ,

CSE DEPARTMENT

SUBMITTED BY –

AVNEET KAUR (9506/22)

GAURI SHARMA (9509/22)

ISHMEET KAUR (9511/22)

CSE 3RD YEAR 6TH SEMESTER

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to everyone who contributed to the successful completion of our project, "TRANSACT RITE."

We would like to sincerely thank DR. SANTOSH KUMAR YADAV for his invaluable guidance, support, and encouragement throughout the development of our project, **TransactRite**. His insights and expertise were instrumental in shaping our work.

We would also like to thank our friends and classmates for their constant support, constructive suggestions, and collaborative spirit, which greatly contributed to the successful completion of this group project.

As a team of three members, working on TransactRite has been an enriching experience, strengthening our technical skills, teamwork, and problem-solving abilities.

Finally, we are deeply grateful to our families and the Almighty for their endless support, patience, and inspiration, which kept us motivated throughout this journey.

AVNEET KAUR (9506/22)

GAURI SHARMA (9509/22)

ISHMEET KAUR (9511/22)

CSE (6TH SEMESTER)

ABSTRACT

In today's digital era, managing personal finances securely and efficiently has become a necessity. TransactRite is a comprehensive Android-based application designed to bring together essential financial services into a single, user-friendly platform.

The app offers secure authentication with email-based password recovery, account creation, and a central dashboard equipped with a side panel featuring a multilingual Chatbot for user assistance. Core functionalities include an Expense Tracker, Currency Converter, Credit Score Monitoring, Transaction History management, Fake Note Detection, and a Fund Transfer Module.

By integrating technologies like Firebase Firestore for real-time data management and Google Speech Recognition API for voice interactions, TransactRite ensures a seamless, secure, and intelligent financial management experience.

The project focuses on accessibility, security, and user empowerment, providing a practical solution for everyday financial activities while maintaining high performance and reliability.

CONTENTS

1) INTRODUCTION

TRANSACT RITE is the product of efforts put in by our prestigious group and team Tech Engineers.

Transact Rite was the perfect choice practically ,technically and user's accepted choice. Lets discuss the aim and objectives on these three grounds briefly,

TRANSACT RITE : a practical choice-

Looking at the advancements of finance and banking services on online portals and platforms it was very much a practical need to bring all these services at single and widely accessible Android platform.

Also with putting in our previous experience with minor project on ATM Simulator being desktop application , Transact Rite was the perfect step-up and choice that uses our knowledge and skillsets to their full potential.

TRANSACT RITE : a technical choice-

Now the very foundation of our team is built around the shared admiration and willingness to work with Java, the most versatile, robust and portable multipurpose language .

All of our team members possess knowledge as well as hands on experience with Java and OOPS concepts and this project in Android platform has provided us all an enriched and suitable advanced learning opportunity.

TRANSACT RITE : a user friendly choice-

Now , technology today is a tool that better serves the masses otherwise the masses need to better the tool !

Bring the financial services and processes at the user friendly and this widely popular choice of platform , Android is using today's technology is used at its finest ! Plus being the consumers of such services we also get to have a close look ,starting from designing going on to the final testing .

1.1 SCOPE

The primary focus of TRANSACT RITE is to provide a highly accessible and user-friendly Android application that offers:

- Easy account management with secure login and password recovery options.
- Centralized Dashboard operations including multiple financial services.
- Integration of a smart Chatbot and support for multiple languages to cater to diverse users.
- Tools for tracking expenses, converting currencies, monitoring credit scores, maintaining transaction histories, detecting fake notes, and transferring funds.

1.2 PURPOSE

The purpose of TRANSACT RITE is to combine multiple essential banking and financial operations into a single application that users can trust.
It aims to enhance convenience, accessibility, and security for users managing their personal finances, all while providing value-added services typically unavailable in conventional banking apps.

1.3 OVERVIEW

TRANSACT RITE offers:

- Secure User Authentication with Forgot Password functionality.
- New Account Creation using basic personal details.
- A centralized Dashboard with:
    - Side Panel Chatbot for assistance.
    - Multi-Language Support.
- Financial Tools:
    - Expense Tracker: Monitor and categorize expenses.
    - Currency Converter: Real-time international rate conversions.
    - Credit Score Monitoring: Track personal credit health.
    - Transaction History: Complete record of financial transactions.
    - Fake Note Detection: Validate currency authenticity.
    - Fund Transfer Module: Seamless transfer of funds between users.

1.4 CORE FEATURES AND FUNCTIONALITIES

1. Authentication Process

- Login Page: Secure login using registered email and password.

- Forgot Password Flow: Sends password reset link to registered email.
- New Account Creation: Users can register with name, email, password

2. Dashboard Features
   - Side Navigation Panel:
     - Quick access to all financial services.
     - Integrated Chatbot for real-time assistance.
     - Multi-Language toggle option.

3. Expense Tracker
   - Record daily expenditures.
   - View expense summary and reports.
   - Categorize spending patterns.

4. Currency Converter
   - Real-time exchange rate updates.
   - Easy conversion between multiple international currencies.

5. Credit Score Monitoring
   - Overview of user's credit rating.
   - Tips to maintain or improve credit scores.

6. Transaction History
   - Displays detailed log of all transactions.
   - Search and filter transactions based on type and date.

7. Fund Transfer Module
   - Secure transfer of funds to another registered user's account.
   - Real-time balance updates post-transfer.

8. Fake Note Detection
   - Uses image processing techniques (Machine Learning if applicable) to verify currency notes.
   - Alerts users to possible counterfeit notes.

2. SOFTWARE DEVELOPMENT LIFE CYCLE(SDLC)

A software life cycle model (also termed process model) is a pictorial and diagrammatic representation of the software life cycle. A life cycle model represents all the methods required to make a software product transit through its life cycle stages. It also captures the structure in which these methods are to be undertaken.

In other words, a life cycle model maps the various activities performed on a software product from its inception to retirement. Different life cycle models may plan the necessary development activities to phases in different ways. Thus, no element which life cycle model is followed, the essential activities are contained in all life cycle models though the action may be carried out in distinct orders in different life cycle models. During any life cycle stage, more than one activity may also be carried out.

NEED OF SDLC

The development team must determine a suitable life cycle model for a particular plan and then observe to it.

Without using an exact life cycle model, the development of a software product would not be in a systematic and disciplined manner. When a team is developing a software product, there must be a clear understanding among team representative about when and what to do. Otherwise, it would point to chaos and project failure. This problem can be defined by using an example. Suppose a software development issue is divided into various parts and the parts are assigned to the team members. From then on, suppose the team representative is allowed the freedom to develop the roles assigned to them in whatever way they like. It is possible that one representative might start writing the code for his part, another might choose to prepare the test documents first, and some other engineer might begin with the design phase of the roles assigned to him. This would be one of the perfect methods for project failure.

A software life cycle model describes entry and exit criteria for each phase. A phase can begin only if its stage-entry criteria have been fulfilled. So without a software life cycle model, the entry and exit criteria for a stage cannot be recognized. Without software life cycle models, it becomes tough for software project managers to monitor the progress of the project.

SDLC Cycle

SDLC Cycle represents the process of developing software. SDLC framework includes the following steps:

The stages of SDLC are as follows:

Stage1: Planning and requirement analysis

Requirement Analysis is the most important and necessary stage in SDLC.

The senior members of the team perform it with inputs from all the stakeholders and domain experts or SMEs in the industry.

Planning for the quality assurance requirements and identifications of the risks associated with the projects is also done at this stage.

Business analyst and Project organizer set up a meeting with the client to gather all the data like what the customer wants to build, who will be the end user, what is the objective of the product. Before creating a product, a core understanding or knowledge of the product is very necessary.

Stage2: Defining Requirements

Once the requirement analysis is done, the next stage is to certainly represent and document the software requirements and get them accepted from the project stakeholders.

This is accomplished through "SRS"- Software Requirement Specification document which contains all the product requirements to be constructed and developed during the project life cycle.

Stage3: Designing the Software

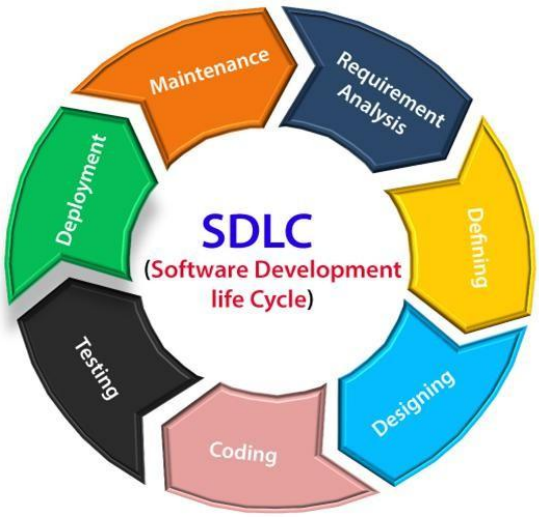
The next phase is about to bring down all the knowledge of requirements, analysis, and design of the software project. This phase is the product of the last two, like inputs from the customer and requirement gathering.

Stage4: Developing the project

In this phase of SDLC, the actual development begins, and the programming is built. The implementation of design begins concerning writing code. Developers have to follow the coding guidelines described by their management and programming tools like compilers, interpreters, debuggers, etc. are used to develop and implement the code.

Stage5: Testing

After the code is generated, it is tested against the requirements to make sure that the products are solving the needs addressed and gathered during the requirements stage.



3) SYSTEM REQUIREMENTS

1. **Hardware Requirements:**
   i. Minimum Desktop Requirements-

   - **Operating System:** Windows 7/8/10 (64-bit), macOS (10.10+), or Linux (GNOME or KDE desktop).
   - **RAM:** Minimum of 4 GB RAM (8 GB RAM or more recommended).
   - **CPU:** Intel Core i5 or higher (recommended), or equivalent AMD processor.
   - **Storage:** Minimum of 2 GB disk space for Android Studio, plus at least 1 GB for Android SDK and emulator system images.
   - **Graphics:** 1280 x 800 minimum screen resolution.

   ii. Minimum Phone Requirements-

   - **Operating System:** Android 8.0 (Oreo).
   - **RAM:** 2 GB (minimum).
   - **CPU:** Quad-core processor.
   - **Storage:** At least 16 GB of internal storage.
   - **Screen Resolution:** 720x1280 pixels (HD).

2. **Software Requirements:**

1. Programming Languages

Java:

Java is a high-level, object-oriented programming language that has been one of the foundational technologies for Android development since its inception. It is known for its robustness, portability, security, and extensive community support. Java follows the **"Write Once, Run Anywhere"** (WORA) principle, which means that compiled Java code can run on any platform that supports the Java Virtual Machine (JVM), making it ideal for mobile development where different hardware and software configurations exist.

- In the **TransactRite** project, Java played a critical role in:
- **Application Logic**: Handling core business logic, backend processes, user input validation, and transaction operations.
- **Activity and Fragment Management**: Managing user interface screens and lifecycle events through Activities, Fragments, and Services.
- **API Integrations**: Facilitating network communications, database transactions, and third-party integrations such as Firebase services.

- **Security Implementations**: Managing authentication workflows, secure data storage, and sensitive transaction handling using Java's secure APIs.

Kotlin:

Kotlin is a modern, statically-typed programming language officially supported by Google for Android app development. Introduced by JetBrains and later adopted by the Android community, Kotlin brings powerful new features that make Android development faster, safer, and more productive compared to Java alone.

- In the **TransactRite** project, Kotlin was a crucial choice for several reasons:
- **Concise Syntax**: Kotlin dramatically reduces boilerplate code. For example, properties, data classes, and event listeners can be written much more cleanly compared to Java, resulting in more readable and maintainable code.
- **Null Safety**: Kotlin's type system is designed to eliminate null pointer exceptions (a common source of bugs in Java applications) at compile time, improving overall app stability and user experience.
- **Coroutines for Asynchronous Programming**: Kotlin provides built-in support for coroutines, which makes it easier to write non-blocking, asynchronous code. This helped the TransactRite app handle background tasks such as network operations and database transactions smoothly without freezing the user interface.
- **Interoperability with Java**: Kotlin is 100% interoperable with Java, meaning existing Java libraries and codebases could be easily reused within the project without any compatibility issues. This allowed gradual migration and integration between Java and Kotlin components.

2. Integrated Development Environment (IDE)

- **Android Studio:**
  The official IDE for Android development, based on IntelliJ IDEA. Android Studio provides a complete development environment including code editing, debugging tools, performance profilers, an emulator, and a robust build system, making it the central hub for coding, testing, and deploying the app.

3. Build Tools

- **Gradle:**
  An advanced and flexible build automation system used to manage project dependencies, compile source code, automate tasks, and customize build configurations for different environments (debug, release).

4. Layout and UI Design

- **XML (Extensible Markup Language):**
  Used to design user interface layouts declaratively, separating the presentation layer from business logic and ensuring a clean architecture.

- **Jetpack Compose:**
  A modern toolkit for building native Android UIs with a declarative programming model. Jetpack Compose simplifies and accelerates UI development, allowing more intuitive and dynamic layouts with less boilerplate code.

5. Android Jetpack Components

- **LiveData:**
  A lifecycle-aware observable data holder that ensures UI components only update when they are active, reducing crashes and memory leaks.
- **ViewModel:**
  Designed to store and manage UI-related data in a lifecycle-conscious way, ensuring data survives configuration changes like screen rotations.
- **Navigation Component:**
  Simplifies the implementation of navigation between different app destinations, providing a clear and consistent user experience.
- **Room Database:**
  An abstraction layer over SQLite, providing an easier and more robust database access while maintaining the full power of SQLite.
- **Data Binding:**
  Allows UI components to bind directly to data sources in the app, reducing boilerplate code and enhancing app performance.

6. Backend and Cloud Services

- **Firebase Authentication:**
  Used for secure, easy-to-implement user authentication, supporting methods like email/password, Google sign-in, and phone number authentication.

- **Firebase Realtime Database:**
  Provides real-time data synchronization across connected clients, ensuring consistent and up-to-date transaction records.
- **Firebase Cloud Firestore (if used):**
  A flexible, scalable NoSQL cloud database for storing user profiles, transactional logs, and related metadata.
- **Firebase Cloud Messaging (FCM):**
  Facilitates the sending of real-time notifications and messages to users, enhancing engagement and communication.

- **Firebase Crashlytics:**
  A powerful crash reporting tool that helps track, prioritize, and fix stability issues in real-time, ensuring a smooth user experience.

7. Debugging and Monitoring Tools

-
  Provides real-time insights into app performance metrics such as CPU, memory, network, and energy usage, helping optimize resource management.

8. Source Control and Collaboration

- **Git:**
  A distributed version control system that tracks changes to the codebase, enables collaboration among developers, and supports efficient branching and merging strategies.

- **GitHub:**
  A cloud-based platform for code hosting, version control, collaboration, and continuous integration/deployment workflows.

9. Design and Prototyping Tools

- **Draw.io:**
  An online tool used for creating flowcharts, UML diagrams, ER models, and other visual documentation crucial for planning app architecture and user flows.

10. Libraries and Dependencies

- **Biometric API**:
  Integrated biometric authentication (fingerprint and facial recognition) for enhanced security.
- **MPAndroidChart**:
  A powerful charting library used to create dynamic and interactive data visualizations within the application.
- **Lottie**:
  An animation library for displaying lightweight, scalable animations to enhance the UI/UX experience.
- **VectorDrawable**:
  Enabled scalable vector graphics to reduce APK size and improve rendering quality.
- **CardView**:
  Implemented card-style UI elements with rounded corners and shadow effects.

- **Preference Library**:
  Simplified the creation of settings and preference screens for better user customization.
- **TensorFlow Lite**:
  Deployed machine learning models on-device for potential intelligent features.
- **OpenCV**:
  Supported real-time computer vision tasks, enhancing image processing capabilities.
- **ML** :
  Enabled on-device machine learning features like text recognition and face detection.

Testing Frameworks:

- **JUnit**:
  Framework for unit testing the core logic and business rules of the application.
- **Espresso**:
  Android UI testing framework for automating interaction with user interface elements to ensure app reliability.

4) ER DIAGRAM FOR THE SYSTEM

Introduction: An ER diagram stands for entity-relationship diagram, which is schematically represented and embodies the data model for a database. In this, the database structure is described by entities, attributes, and relationships. The biggest application of ER diagrams is at the design stage of database development, making sure that all the related data has been captured and properly structured.
Elements of an ER Diagram:

- Entities:
  - Definition: Entities are real world objects or things with existence independent from the database. They are real-world entities like Students, Courses, and Instructors, etc.
  - Notation: Typically shown as rectangles with the name of the entity inside.
  - Attributes: The properties or characteristics of entities. Attributes are represented as ovals with lines connecting them to the relevant entities. These include StudentID, Name, and DateOfBirth, for instance.
- Relationships:
  - Definition: An informal way to describe what the entities relate each other. For instance, a Student "enrolls in" a Course.
  - Notation: Represented with diamonds with the relationship name inside connected with lines to the entities.
  - Cardinality: The number of occurrences of one entity which are allowed to be associated with occurrences of another. One-to-one, one-to-many, many-to- many.

- Attributes:
  - Definition: The properties that define an entity or of its relations in a database are referred to as attributes. They also define the student entity. For example, name, email, and date of birth can be considered as attributes of Student entity.
- Types of Attributes:
  - Simple vs. Composite: A simple attribute is an attribute that cannot be further broken down (like Age). This means, in plain words, that composite attributes can be decomposed into sub-attributes. For example, Name can be a composite attribute, and

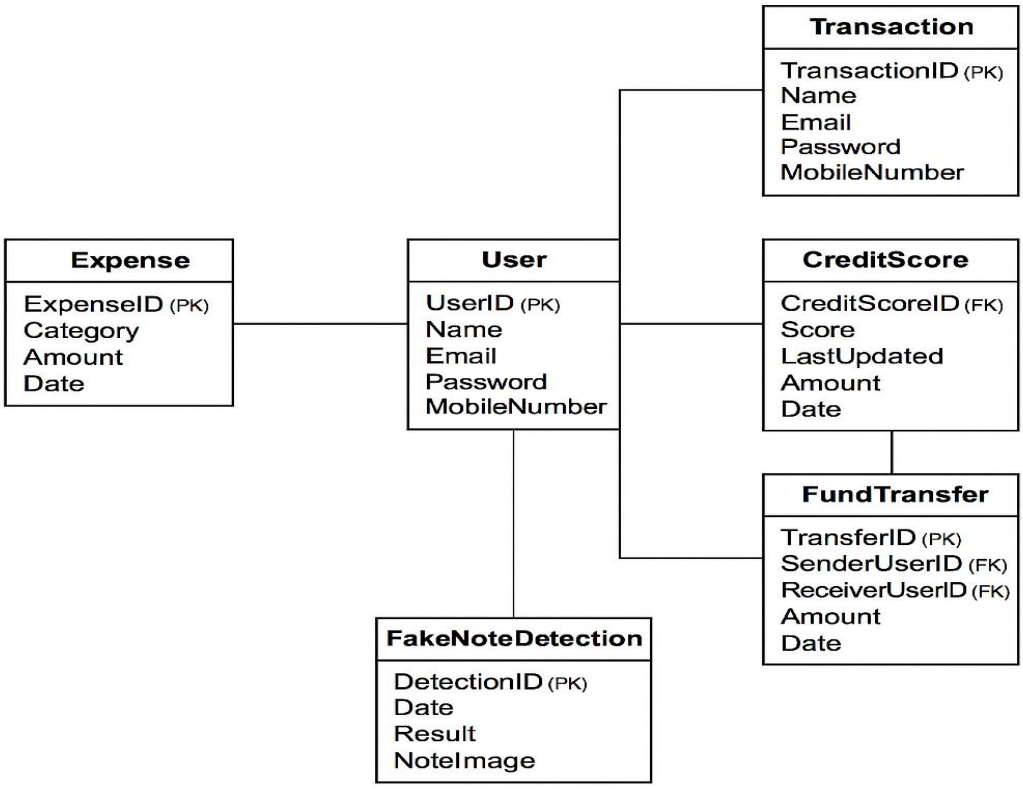
the sub-attributes would be FirstName and LastName.

- Single-Valued vs. Multi-Valued: Single-valued attributes carry just one value, as in Social Security Number. Multi-valued attributes carry multiple values. An example of multi-valued attributes is PhoneNumbers.
- Attributes that are derivable from other attributes (example: Age is derivable from DateOfBirth).

- Primary Keys:
  - Definition: It is an attribute or a set of attributes in an entity which uniquely identifies each instance of the same.
  - Notation: Underlined in the entity rectangle.
- Foreign Keys:
  - Definition: A foreign key is that attribute in an entity which links up with the primary key of another entity in order to link the two entities.

ER Diagram Development:

- Identify key entities to be included in your database.
- Specify the relationships between entities, including its type and cardinality.
- Specify attributes for each entity, and you will determine which of the attributes will serve your primary and foreign keys .
- Utilize diagramming software or tool to visualize entities, relationships.



5) DATA FLOW DIAGRAMS FOR THE SYSTEM

Data Flow Diagram (DFD): It illustrates the flow of data in a system with a diagram. It shows how the system converts its inputs into outputs, which would outline a general framework and activities in the system. DFDs are mainly employed at the front end of the system design to collect data flow and look for areas where improvement is needed.

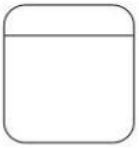
Elements of a DFD:

- Processes:
  - ❖ Definition: Processes are activities or functions that are used to transform data in the system.
  - ❖ Notation: Usually represented as circles or rounded rectangles.
  - ❖ Examples: Register Student, Process Payment, Generate Report.

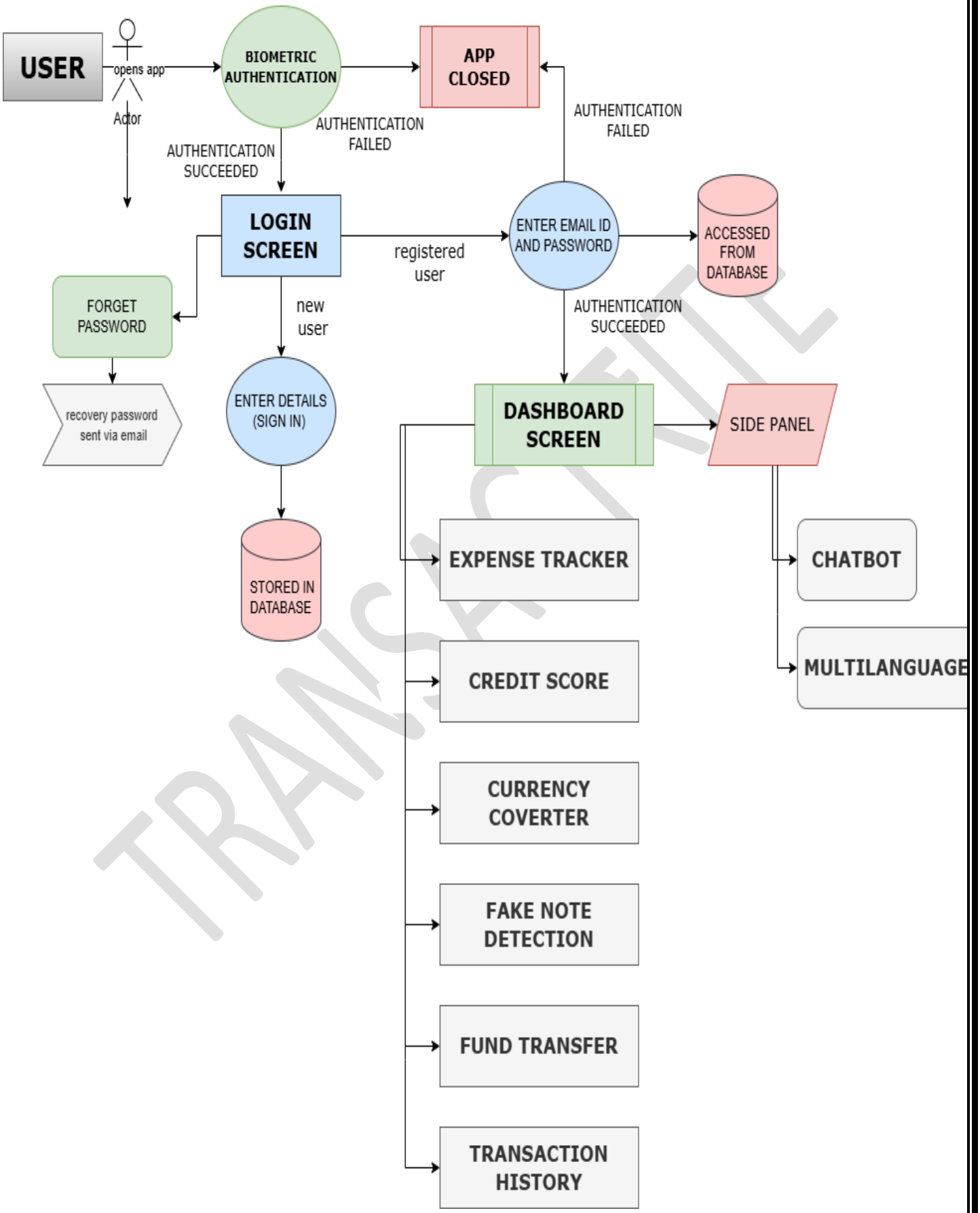
- Data Stores:
  - ❖ Definition: Data stores are those repositories where data is accumulated and kept for use by processes.
  - ❖ Notation: Usually represented as open-ended rectangles or parallel lines.
  - ❖ Examples: Student Database, Course Records, Payment History.

- Data Flows:
  - ❖ Definition: Data flows depict the movement of data between processes, data stores, and external entities.
  - ❖ Notation : Represented using arrows of the directions of data flowing.
  - ❖ Examples: Student Details, Payment Accounts, Course Details.

DFD Symbols and Definitions

Process		• Process - performs some action on data, such as creates, modifies, stores, delete, etc. Can be manual or supported by computer.
Data store		• Data store - information that is kept and accessed. May be in paper file folder or a database.
External Entity		• External entity - is the origin or destination of data. Entities are external to the system.
Data flow		• Data flow - the flow of data into or out of a process, datastore or entity

DATA FLOW DIAGRAM OF THE COMPLETE SYSTEM



6) MODULES

6.1) Login Screen Module

---

1. Introduction

1.1 Purpose

This module ensures secure user authentication using **email/password login**, **fingerprint authentication**, and **Firebase Authentication**.

1.2 Scope

The **Login Screen Module** is a critical component of the banking application, ensuring secure access to user accounts. It will include:

- **Email and password authentication** using Firebase
- **Fingerprint authentication (biometric login)** for quick access
- **Secure storage of login sessions** to maintain authentication state
- **Forgot Password feature** for password recovery
- **Two-factor authentication (2FA) support** (optional for additional security)
- **Material Design UI** for an intuitive user experience

The login module enhances security, ensuring that only verified users can access banking features.

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- **Developers** – To implement the login module
- **Testers** – To validate authentication features
- **Project Managers** – To oversee development and integration
- **Stakeholders** – To understand security mechanisms in the banking app

1.4 References

- Firebase Authentication Documentation
- Android Biometric Authentication Guide
- Material Design Guidelines

2. Overall Description

2.1 Product Perspective

The **Login Screen Module** is an essential security layer of the banking app, integrating with:

- **Firebase Authentication** for user identity management
- **Biometric Authentication API** for fingerprint-based login
- **Room Database (if needed)** for local session storage
- **Encrypted SharedPreferences** for securely storing authentication tokens

2.2 Product Functions

The login module will include:

- **User Registration** – New users can sign up with email and password
- **User Login** – Existing users can log in with credentials
- **Fingerprint Login** – Users can authenticate using biometrics
- **Forgot Password** – Allows users to reset their password via email
- **Session Management** – Keeps users logged in securely
- **Logout Functionality** – Allows users to sign out securely
- **Multi-Factor Authentication (MFA) (Optional)** – Enhances security for high-value transactions

2.3 User Characteristics

- **Bank Customers** – Users with an account in the banking system
- **Security-Conscious Users** – Users who prefer biometric authentication for security
- **General Mobile Users** – Users with basic knowledge of mobile banking apps

3. Specific Requirements

3.1 Functional Requirements

ID	Requirement	Description
FR-1	User Registration	Users can register using email & password via Firebase

FR-2	Email/Password Login	Users can log in using Firebase Authentication
FR-3	Fingerprint Login	Users can log in using biometric authentication
FR-4	Forgot Password	Users can reset their password via email verification
FR-5	Session Management	Users remain logged in unless they manually log out
FR-6	Logout	Users can securely log out of their account
FR-7	Multi-Factor Authentication (MFA)	Optional feature for enhanced security

3.2 Non-Functional Requirements

ID	Requirement	Description
NFR-1	Security	User credentials must be securely stored using Firebase Authentication and encrypted storage
NFR-2	Performance	Login authentication should take no more than 2 seconds
NFR-3	Usability	The UI must follow Material Design principles for a seamless user experience
NFR-4	Compatibility	The module must work on Android 8.0 (Oreo) and above

---

4. External Interface Requirements

4.1 User Interfaces

- **Login Screen:** Fields for email/password, fingerprint login button, and "Forgot Password" link
- **Registration Screen:** Fields for email, password, and confirm password
- **Error Messages:** Proper validation messages for incorrect inputs

4.2 Software Interfaces

- **Firebase Authentication** for user login and registration
- **Android Biometric API** for fingerprint authentication
- **Room Database (if needed)** for local storage of session data

4.3 Hardware Interfaces

- **Fingerprint Sensor** (Required for biometric authentication)

---

5. Constraints

- The app must have an active internet connection for Firebase Authentication.
- Fingerprint login requires a device with a fingerprint sensor.
- The app must follow banking security compliance guidelines.

---

6. Assumptions and Dependencies

- Users have a registered banking account.
- The device supports **Google Play Services** for Firebase Authentication.
- Biometric authentication is enabled in the device settings.

6.2) Dashboard Screen & Button Functionality

---

1. Introduction

1.1 Purpose

The Dashboard Screen is a core component of the Banking App, providing users with quick access to key financial services. This module ensures an intuitive and responsive interface with seamless navigation to essential banking functionalities, including account management, transactions, bill payments, and user notifications.

1.2 Scope

This module will:

- Display account summary and balance information.
- Provide quick access to key banking features via buttons.
- Ensure smooth user interactions with responsive UI elements.
- Implement security measures for safe financial transactions.
- Log user interactions for enhanced tracking and analytics.

1.3 Overview

This document outlines the functional and non-functional requirements, software dependencies, and constraints for the Dashboard Screen and its button functionalities.

---

2. Functional Requirements

2.1 Dashboard UI Design

- The app shall display the user's account summary, including balance and transaction history.
- The UI shall contain quick-access buttons for essential banking features.
- The dashboard shall have a notification section for alerts and updates.

2.2 Button Functionalities

- **Money Transfer**: Opens the fund transfer module with input fields for recipient details and amount.
- **Bill Payments**: Provides access to utilities, mobile recharge, and other payment services.
- **Transaction History**: Displays a categorized view of past transactions.
- **Credit Score Check**: Opens the credit score module for analysis and report generation.
- **Fake Note Detection**: Launches the module for detecting counterfeit currency using image processing.
- **Voice Control**: Enables voice-based navigation and transaction execution for accessibility.

2.3 User Feedback & Logging

- The app shall provide instant feedback on successful or failed actions.
- All dashboard interactions shall be logged securely for tracking purposes.
- Users shall receive error messages in case of connectivity or processing issues.

---

3. Non-Functional Requirements

3.1 Performance

- The dashboard shall load within 2 seconds.

- Button responses shall execute actions within 1 second.

3.2 Compatibility

- The module shall be compatible with Android 8.0 (API 26) and above.
- It must function on different screen sizes and orientations.

---

4. Software & Hardware Requirements

4.1 Software Requirements

- Android Studio Ladybug 2024.2.2
- Kotlin DSL for build configurations
- Gradle 8.8 or higher
- Firebase for authentication and transaction logging

4.2 Hardware Requirements

- **Minimum Device Specs:**
  - RAM: 2GB+
  - Screen: 5.5"+
  - CPU: ARM 64-bit architecture
  - Storage: 500MB available

---

5. Constraints & Limitations

- Real-time transactions depend on internet connectivity.
- Processing speed may vary based on device hardware.
- The first version supports Indian banking services only.

---

6. Future Enhancements

- AI-powered financial recommendations.
- Integration with UPI and international payments.
- Cloud-based fraud detection system.

---

7. References

- Android Material Design Guidelines
- Firebase Authentication & Realtime Database

6.3) Expense Tracker Module

1.Introduction

1.1 Purpose

The Expense Tracker Module is a personal finance management feature within the TransactRite application. It enables users to set a weekly budget, add categorized expenses, visualize spending through pie charts, and monitor budget adherence in real-time. The module encourages better financial planning by offering intuitive tracking and reporting features.

1.2 Scope

This module will:

- Allow users to set and adjust a weekly budget.
- Add daily expenses categorized under predefined or custom categories.
- Visualize spending distribution using a pie chart.
- Summarize weekly spending and highlight over/under-budget performance.
- Identify the highest spending category to help users understand spending patterns.

1.3 Overview

This section outlines the functional and non-functional requirements, software dependencies, constraints, and future enhancement possibilities for the Expense Tracker Module.

---

2. Functional Requirements

2.1 Budget Setup

- Users can input a weekly budget amount.
- The app shall validate that the budget value is not empty before proceeding.

2.2 Expense Entry

- Users can input an expense amount and select a category from a dropdown list.
- If "Custom" is selected, users can define their own category.
- Expenses are timestamped upon addition.

2.3 Weekly Summary Generation

- The app computes the total amount spent within the last 7 days.
- Displays whether the user is under or over the set budget.
- Identifies and displays the highest spending category.

2.4 Visual Analytics

- A pie chart shall dynamically represent category-wise expense distribution.
- The pie chart updates automatically after each expense addition.

2.5 Data Management

- Expenses are maintained in a local in-memory list during runtime.
- RecyclerView shall list all recorded expenses with relevant details.

---

3. External Interface Requirements

3.1 User Interfaces

- Input fields for Weekly Budget, Expense Amount, and Custom Category (optional).
- Dropdown (Spinner) for selecting predefined or custom categories.
- RecyclerView to display all expenses.

- PieChart for visual analytics.
- Weekly budget summary TextView.

3.2 Software Interfaces

- MPAndroidChart library for pie chart visualization.
- Android RecyclerView for dynamic list management.

3.3 Hardware Interfaces

- None specific beyond standard Android device requirements.

---

4. Constraints

- Current version stores data temporarily during app session (no database persistence).
- Expense tracking is limited to weekly analysis (7-day window).

---

5. Assumptions and Dependencies

- Users will consistently enter valid numeric data for budgets and expenses.
- Application requires sufficient RAM to handle dynamic RecyclerView and PieChart updates.

---

6. Future Enhancements

- Persistent storage using Room Database or Firebase Firestore.
- Monthly and yearly spending summaries.
- Advanced analytics (spending trends, forecast).
- Notifications for nearing budget limits.
- Export expense reports as PDF.

6.4) Credit Score Calculator Module (Financial Analysis Module)

---

1. Introduction

1.1 Purpose

The Credit Score Calculator Module is an essential part of the financial analysis feature in the existing Android application. It enables users to enter financial details and calculates an estimated credit score based on predefined criteria. The module provides a user-friendly UI, performs real-time computations, and presents the results instantly.

1.2 Scope

This module will:

- Be triggered by a button within the main application.
- Allow users to input financial details such as credit utilization, loan history, and inquiries.
- Compute an estimated credit score using predefined weightage factors.
- Display the computed credit score dynamically on the UI.
- Optionally store results in a local database for future reference.

1.3 Overview

This section outlines the functional and non-functional requirements, software dependencies, and constraints for the Credit Score Calculator Module.

---

2. Functional Requirements

2.1 User Input Collection

- The module shall collect the following user inputs:
    - Total Credit Limit
    - Credit Used
    - Age of Credit (Months)
    - Number of Loan Accounts
    - Number of Hard Inquiries
    - Payment History (Good, Late, Missed)

2.2 Credit Score Calculation

- The system shall calculate the credit score based on the following weightage:
    - **Payment History**: 35%
    - **Credit Utilization**: 30%
    - **Credit Age**: 15%
    - **Credit Mix (Loan Accounts)**: 10%
    - **Hard Inquiries**: 10%

2.3 Score Display

- The app shall display the computed credit score on the screen dynamically.
- The UI shall provide real-time updates based on user inputs.

2.5 User Feedback & Logging

- The app shall provide suggestions for improving credit scores based on user inputs.
- The app shall maintain a log of score calculations if database storage is enabled.

---

3. Non-Functional Requirements

3.1 Performance

- Credit score computation shall take no more than 1 second.
- The module shall work efficiently on devices with 2GB RAM and above.

3.2 Compatibility

- The module shall be compatible with Android 8.0 (API 26) and above.
- It must work on various Android screen sizes and orientations.

3.3 Security & Privacy

- User data shall be processed locally without exposure to external servers.
- If Firebase is enabled, user consent shall be required before data storage.

3.4 Scalability

- The module should allow future enhancements such as credit report visualization and third-party API integration.
- Support for multi-source financial data inputs in future versions.

---

4. Software & Hardware Requirements

4.1 Software Requirements

- Android Studio Ladybug 2024.2.2
- Kotlin DSL for build configurations
- Gradle 8.8 or higher
- SQLite for local storage (optional)
- Firebase Firestore (optional for cloud storage)

4.2 Hardware Requirements

- **Minimum Device Specs:**
  - **RAM**: 2GB+
  - **Storage**: 200MB available
  - **CPU**: ARM 64-bit architecture

---

5. Constraints & Limitations

- The accuracy of the estimated credit score depends on user-provided data.
- The module does not provide an official credit score but an estimated value.
- Future API integration may require an internet connection.

---

6. Future Enhancements

- Credit report PDF export functionality.
- Integration with third-party credit score APIs for accuracy enhancement.
- Improved UI with graphical score analysis and recommendations.

---

7. References

- Android SQLite Documentation
- Firebase Firestore Guide
- Financial Credit Score Calculation Standards

6.5) Currency Converter Module

---

1. Introduction

1.1 Purpose

The Currency Converter Module is a utility feature within the TransactRite application that enables users to easily convert amounts between multiple international currencies. It provides fast, offline conversion based on predefined exchange rates for user convenience during transactions and financial planning.

1.2 Scope

This module will:

- Allow users to select a source currency and a target currency.
- Accept a user-entered amount to be converted.
- Perform conversion based on internally defined exchange rates.
- Display the converted amount instantly.
- Handle invalid inputs gracefully.

1.3 Overview

This section defines the functional and non-functional requirements, external interfaces, constraints, assumptions, and future enhancement plans for the Currency Converter Module.

---

2. Functional Requirements

2.1 Currency Selection

- The app shall provide two dropdown menus (Spinners) for selecting 'From' and 'To' currencies.
- Supported currencies: USD, INR, EUR, GBP, JPY.

2.2 Amount Input

- Users can input the amount to be converted using an EditText field.
- Input validation must ensure only numeric entries are processed.

2.3 Currency Conversion

- The system shall first normalize the input amount to USD and then convert it to the target currency.
- Conversion shall be based on predefined fixed rates stored locally in the application.

2.4 Result Display

- The app shall dynamically display the converted amount rounded to two decimal places.
- In case of invalid inputs, an appropriate error message shall be shown.

---

1. *Non-Functional Requirements*

ID	Requirement	Description
NFR-1	Performance	Conversion and result display must occur within 500 milliseconds.
NFR-2	Usability	Simple and intuitive UI with minimal user interaction steps.
NFR-3	Compatibility	The module must support Android 8.0 (API 26) and higher.

NFR-4	Reliability	Error handling for empty or invalid amount inputs must be implemented.

---

4. External Interface Requirements

4.1 User Interfaces

- Two Spinners for currency selection (From and To).
- EditText for inputting the amount.
- Button for triggering conversion.
- TextView for displaying results or error messages.

4.2 Software Interfaces

- No external API calls; operates completely offline.

4.3 Hardware Interfaces

- No special hardware requirements beyond basic Android smartphone capabilities.

---

5. Constraints

- Exchange rates are static and hardcoded; they do not reflect real-time market fluctuations.
- Supports only five currencies in the current version.

---

6. Assumptions and Dependencies

- Users will manually update the app for revised currency rates if needed.

- The device should allow basic UI interactions without system limitations.

---

7. Future Enhancements

- Integration with live Forex APIs (e.g., Open Exchange Rates, Fixer.io) for real-time rates.
- Addition of more currencies and cryptocurrency support.
- Historical conversion data visualization.
- Currency trend charts and financial news integration.

6.6) FAKE NOTE DETECTION (Image Processing Module (OpenCV))

---

1. Introduction

1.1 Purpose

The Image Processing Module is a crucial part of the Fake Note Detection module of TransactRite android finance system. It leverages OpenCV to process images captured from the device's camera, analyze currency notes, and classify them as **genuine** or **counterfeit**.

1.2 Scope

This module will:

- Capture images using CameraX.
- Preprocess images (grayscale conversion, edge detection, noise removal).
- Extract key features from currency notes using OpenCV techniques.
- Use machine learning models (TensorFlow Lite) to classify currency notes.
- Display results and provide user feedback.

1.3 Overview

This document outlines the functional and non-functional requirements, software dependencies, and constraints for the Image Processing Module.

---

2. Functional Requirements

2.1 Image Acquisition

- The app shall use the device camera to capture currency note images.
- The image capture process shall use **CameraX** for better control over focus and exposure.

2.2 Image Preprocessing

- Convert captured images to **grayscale**.
- Apply **Gaussian Blur** to reduce noise.
- Perform **Edge Detection (Canny)** to highlight currency features.
- Extract **Regions of Interest (ROI)** such as watermarks, security threads, and serial numbers.

2.3 Feature Extraction

- Extract **Key Points** using ORB (Oriented FAST and Rotated BRIEF) or SIFT.
- Perform **Histogram Analysis** to differentiate genuine and fake notes.
- Use **Contour Detection** to analyze note patterns.

2.4 Fake Note Classification

- The app shall integrate with **TensorFlow Lite (TFLite)** for real-time classification.
- Based on extracted features, classify the note as **genuine** or **fake**.
- Display the classification result on the UI.

2.5 User Feedback & Logging

- If a fake note is detected, the app shall alert the user.
- The app shall store scan logs in local storage for future reference.

---

3. Non-Functional Requirements

3.1 Performance

- Image processing should take less than **2 seconds**.
- The module shall work on devices with **2GB RAM and above**.

3.2 Compatibility

- The module shall be compatible with **Android 8.0 (API 26) and above**.
- It must work on various Android screen sizes and orientations.

3.3 Security & Privacy

- Captured images shall not be stored permanently.
- User data shall not be shared without consent.

3.4 Scalability

- The module should allow future integration of advanced AI models.
- It should support multi-currency detection in future versions.

---

4. SoftwareRequirements

4.1 Software Requirements

- **Android Studio Ladybug 2024.2.2**
- **Kotlin DSL for build configurations**
- **Gradle 8.8 or higher**
- **OpenCV 4.7.0 for Android**
- **TensorFlow Lite for ML integration**

---

5. Constraints & Limitations

- Real-time detection may vary in low-light conditions.
- Processing speed depends on the device hardware.
- Only supports **Indian currency notes** in the current version.

---

6. Future Enhancements

- Support for multiple currency types.
- Cloud-based fraud detection API integration.
- Improved deep learning models for accuracy enhancement.

---

7. References

- OpenCV Documentation
- Android CameraX Guide
- TensorFlow Lite

6.7) Fund Transfer Module

---

1. Introduction

The Fund Transfer module is a crucial component of a financial application that enables users to securely manage their funds across multiple accounts. It supports functionalities such as user registration, transaction management (deposit and withdrawal), fund transfer to other users, and voice-controlled operations for enhanced accessibility.
This module uses Firebase Firestore as the database backend and integrates Google Speech Recognition API for voice interaction, ensuring secure, real-time, and user-friendly services.

---

2. Scope

The module focuses on key aspects of personal banking, particularly:

- User Registration: Secure onboarding of users by collecting essential details.
- Transaction Management: Depositing and withdrawing funds from personal accounts.
- Fund Transfers: Sending funds to other registered users.
- Voice Control: Allowing users to perform actions through voice commands.

The scope primarily covers individual user account management and fund movements in a closed network environment.

---

3. Purpose

The Fund Transfer module is designed to:

- Provide a secure and reliable system for basic banking operations.

- Simplify transactions with intuitive UI and voice-based commands.
- Enable quick and efficient movement of funds between accounts.
- Enhance user experience through real-time feedback and database synchronization.

---

4. Overview

The module is comprised of four major functionalities:

- **User Registration**: Collecting user data and initializing accounts using **Firebase Firestore**.
- **Transaction Management**: Managing deposits and withdrawals with instant balance updates.
- **Fund Transfer**: Securely transferring money to other registered accounts.
- **Voice Control**: Executing actions through voice commands via **Google Speech Recognition API**.

Data consistency and efficient transaction handling are maintained using **Firestore's real-time data management** features.

---

5. Functional Requirements

5.1 *Fund Transfer Functionalities*

- User Registration: Name, email, account number, and balance.
- Deposit/Withdrawal: Real-time fund management for a single account.
- Fund Transfer: Ability to send funds to another user by specifying the recipient's account number.
- Voice Operations: Voice-triggered deposit, withdrawal, and transfer operations.

---

6. Non-Functional Requirements

- Security: Data stored and retrieved securely from Firestore.
- Performance: Quick voice recognition and minimal delay in transaction execution.

- Scalability: Designed to handle a growing number of users and transactions efficiently.
- Usability: Simple and intuitive UI with seamless navigation.
- Reliability: Ensuring accurate processing of all user actions without errors.

---

7. Hardware and Software Requirements

7.1 *Hardware Requirements*

- Android smartphone (API Level 21 and above)
- Minimum 2GB RAM, Recommended 4GB+
- Internet connection for Firebase access

7.2 *Software Requirements*

- Android Studio (for development and testing).
- Firebase Firestore (for data storage and retrieval).
- Google Speech Recognition API (for voice control).
- Internet Connectivity (for real-time data synchronization).

---

8. Constraints and Limitations

- Dependent on stable internet connectivity for Firestore operations.
- Voice recognition accuracy may be affected by background noise.
- Firestore free-tier limitations (e.g., reads, writes) could impact heavy transaction load.
- Requires user account number to be known for sending money.

---

9. Future Enhancements

9.1 *Integration with Real-Time Payment Gateway*

- Add support for third-party gateways like **PayPal**, **UPI**, or **Stripe**.
- Implement OTP (One-Time Password) verification for fund transfers.
- Enable bank-to-bank transfers using APIs from official banking institutions.

9.2 *Improved UI/UX Features*

- Quick action buttons for deposits, withdrawals, and transfers.
- Graph-based transaction history visualization.
- Enhanced voice feedback confirming successful transactions.

With these future enhancements, the module can be extended to a full-fledged digital payment system.

---

10. Key Technologies Used

1. **Firebase Firestore Database**: For real-time, scalable, and secure data storage.

2. **Google Speech Recognition API**: For hands-free voice command functionality.

3. **Android (Java/Kotlin)**: For application development.

6.8) Transaction History Module - Transact₹ite App

---

1. Introduction

The Transaction History module in the Transact₹ite app enables users to view a comprehensive record of their financial activities including deposits, withdrawals, and fund transfers. This module ensures transparency, accountability, and ease of access to all past transactions, forming a core part of the user's banking experience.

---

2. Scope

The module is designed to display all transactions linked to the user's account in a simple and user-friendly manner. It supports the viewing of transaction details like date, amount, type, and status. Future updates will allow users to filter, search, and export their transaction history for personal finance management or official use.

3. Purpose

- To provide users with easy access to a detailed record of their past transactions.
- To promote transparency in financial dealings.
- To help users track their financial behavior over time.
- To assist users in reconciling their personal records with the app's transaction logs.

---

4. Functional Requirements

- The system must retrieve transaction records from the Firestore database.
- The system must display transactions sorted by date, with the latest first.
- Each transaction record must show:
    - Transaction type (Deposit, Withdrawal, Transfer)
    - Date and time of transaction
    - Transaction amount
    - Transaction status (Success, Pending, Failed — currently placeholder is null)
- The system must dynamically update the transaction list when a new transaction occurs.

---

5. Non-Functional Requirements

- The module must load transaction history within 2 seconds for up to 1000 transactions.
- The user interface must be responsive and adapt to all screen sizes (phones, tablets).
- Data fetched must be encrypted during transmission to ensure security.
- Transaction data must be displayed in a clean and readable format with appropriate use of font sizes, icons, and colors.
- System downtime for transaction history retrieval must be less than 1% monthly.
- App must handle database connection failures gracefully with a friendly error message.

---

6. Technologies Used

- **Frontend**: Java (Android)
- **Backend**: Firebase Firestore Database
- **Authentication**: Firebase Authentication
- **UI Frameworks**: Android XML layouts with Material Design principles

---

7. Current Limitations

- Transaction status currently appears as null instead of meaningful values like *Success* or *Failed*.
- No filtering, searching, or exporting options implemented yet.
- Limited to basic transaction information without additional metadata (e.g., transaction remarks, recipient details).

---

8. Future Scope

- Implement dynamic and real transaction statuses.
- Introduce advanced search and filter functionalities.
- Enable downloadable transaction reports.
- Integration with notifications to alert users for each completed transaction.

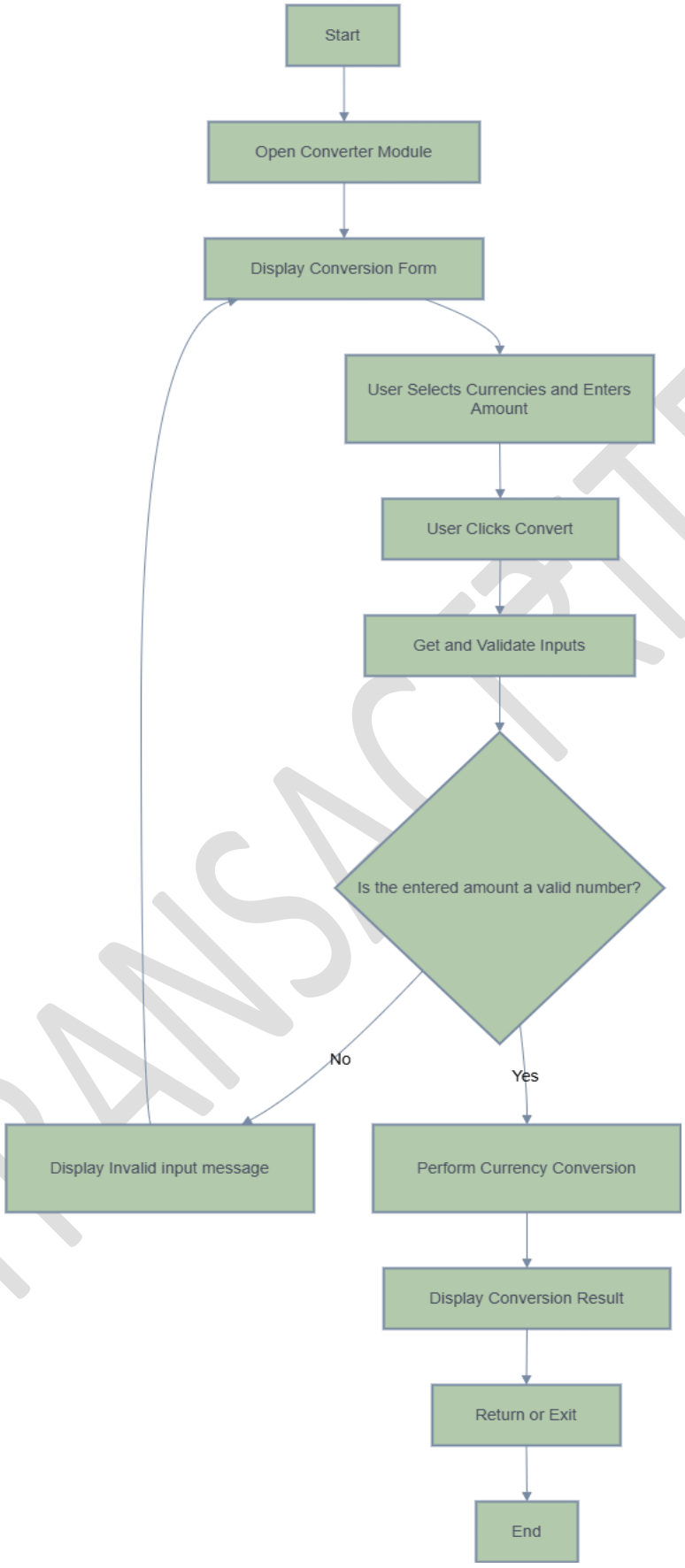
8.FLOWCHARTS

1. Currency Converter Module Flowchart

- **Description:** This flowchart illustrates the sequence of operations performed when a user converts an amount from one currency to another.
- **Flowchart Steps:**
    1. **Start**
    2. **Initialize:**
        - Get references to UI elements: Spinners (fromCurrency, toCurrency), EditText (amountInput), Button (btnConvert), TextView (resultText).
        - Define currency conversion rates (USD, INR, EUR, GBP, JPY).
        - Populate the fromCurrency and toCurrency Spinners with currency options.
    3. **User Action:** User clicks the "Convert" button (btnConvert).
    4. **Get Input:**
        - Retrieve the selected "from" currency from fromCurrency.
        - Retrieve the selected "to" currency from toCurrency.
        - Get the amount to convert from amountInput and convert it to a double.
    5. **Validate Input:**
        - Check if the amount input is a valid number.
        - If invalid, display "Invalid input. Please enter a valid amount." in resultText and go to **End**.
    6. **Get Conversion Rates:**
        - Call getRate(from) to get the conversion rate for the "from" currency.
        - Call getRate(to) to get the conversion rate for the "to" currency.
    7. **Perform Conversion:**
        - Calculate amountInUSD = amount / fromRate.

- Calculate convertedAmount = amountInUSD * toRate.

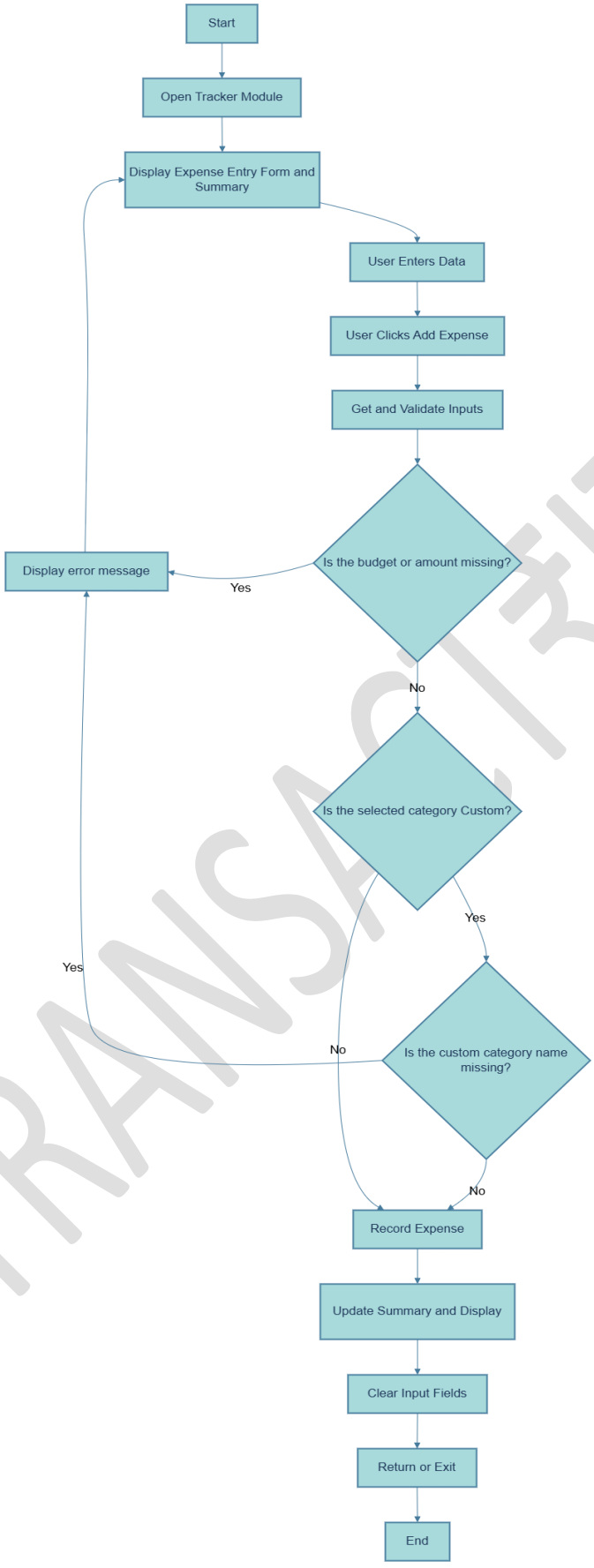
8. **Display Result:**
    - Format the convertedAmount and display it in resultText.
9. **End**



CURRENCY CONVERTER FLOWCHART

2. Expense Tracker Module Flowchart

- **Description:** This flowchart illustrates the process of adding a new expense and updating the expense summary in the Expense Tracker.
- **Flowchart Steps:**
    1. **Start**
    2. **Initialize:**
        - Get references to UI elements: EditTexts (editTextBudget, editTextAmount, editTextCustomCategory), Spinner (spinnerCategory), Button (btnAddExpense), TextView (textViewSummary), RecyclerView (recyclerView), PieChart (pieChart).
        - Initialize expenseList (List of Expense objects).
        - Set up the spinnerCategory with default categories (Food, Transport, etc.).
        - Set up the recyclerView with expenseAdapter.
        - Set up the pieChart.
    3. **User Action:** User clicks the "Add Expense" button (btnAddExpense).
    4. **Get Input:**
        - Get the weekly budget from editTextBudget and convert it to a double.
        - Get the expense amount from editTextAmount and convert it to a double.
        - Get the selected category from spinnerCategory.
        - If the selected category is "Custom", get the custom category name from editTextCustomCategory.
    5. **Validate Input:**
        - Check if the weekly budget and amount are valid numbers and not empty.
        - If invalid, display a "Please enter both budget and amount" Toast message and go to **End**.
        - If the category is "Custom", check if the custom category name is not empty. If empty, display "Enter custom category" Toast message and go to **End**.
    6. **Create Expense Object:**
        - Create a new Expense object with the amount, category, and current timestamp.
    7. **Add Expense to List:**
        - Add the new Expense object to the expenseList.
        - Notify the expenseAdapter that the data has changed.
    8. **Clear Input Fields:**
        - Clear the editTextAmount and editTextCustomCategory.
    9. **Update Summary:**
        - Call updateSummary() to:
            - Calculate the total expenses for the current week.
            - Compare the total expenses with the weekly budget.
            - Determine the category with the highest spending.
            - Update the textViewSummary with the calculated information.
            - Update the pieChart to visualize the spending distribution.
    10. **End**



EXPENSE TRACKER FLOWCHART

3. Fake Note Detection Module Flowchart

• **Description:** This flowchart illustrates the process of capturing a note image, analyzing the note for text, color, and security thread verification, and displaying the results to determine whether the currency note is genuine or possibly counterfeit.

• Flowchart Steps:

1. Start

2. Initialize:

- Initialize OpenCV and ML Kit libraries.
- Set up references to UI elements:
  - ImageView (imageView, debugImageView)
  - EditText (serialNumberInput)
  - TextViews (resultTextView, resultTextView2, resultTextView3)
  - Buttons (btnAnalyze, btnBack, btnViewRecognizedText).
- Retrieve image URI passed from previous activity (ImagePreviewActivity).
- Load and display the captured image in imageView.

3. User Action:

- User enters the serial number in the input field (optional).
- User clicks the **Analyze** button (btnAnalyze).

4. Process Image:

- Load the bitmap from the given image URI.
- Convert the bitmap into OpenCV Mat format.
- Preprocess the image by:
  - Converting to grayscale.
  - Applying adaptive thresholding for text clarity.

5. Perform Text Recognition (OCR):

- Pass the preprocessed grayscale image to ML Kit Text Recognition.
- Extract text from the note.
- Normalize the recognized text:
  - Correct OCR errors.
  - Remove serial number if provided.
  - Standardize common phrases.

6. Validate Authentication:

- Check if recognized text contains:
  - "Reserve Bank of India"
  - "Central Government"

- If found, mark text authentication as successful; otherwise, mark as failed.

7. Identify Denomination:

- Detect denomination keywords in recognized text:
  - ₹10, ₹20, ₹50, ₹100, ₹200, ₹500, ₹2000.
- If denomination detected, proceed to color verification.
- If denomination not detected, display warning.

8. Verify Note Color:

- Analyze the ink color of the note based on its denomination using HSV color space.
- Compare detected color with expected color ranges.
- If color matches expected range, mark as genuine ink; otherwise, possible counterfeit.

9. Detect Security Thread:

- Apply contour detection to find tall, thin, vertical features (reflective thread).
- Measure aspect ratio and alignment of detected contours.
- If thread segments detected and aligned vertically, mark thread detection successful; otherwise, failed.

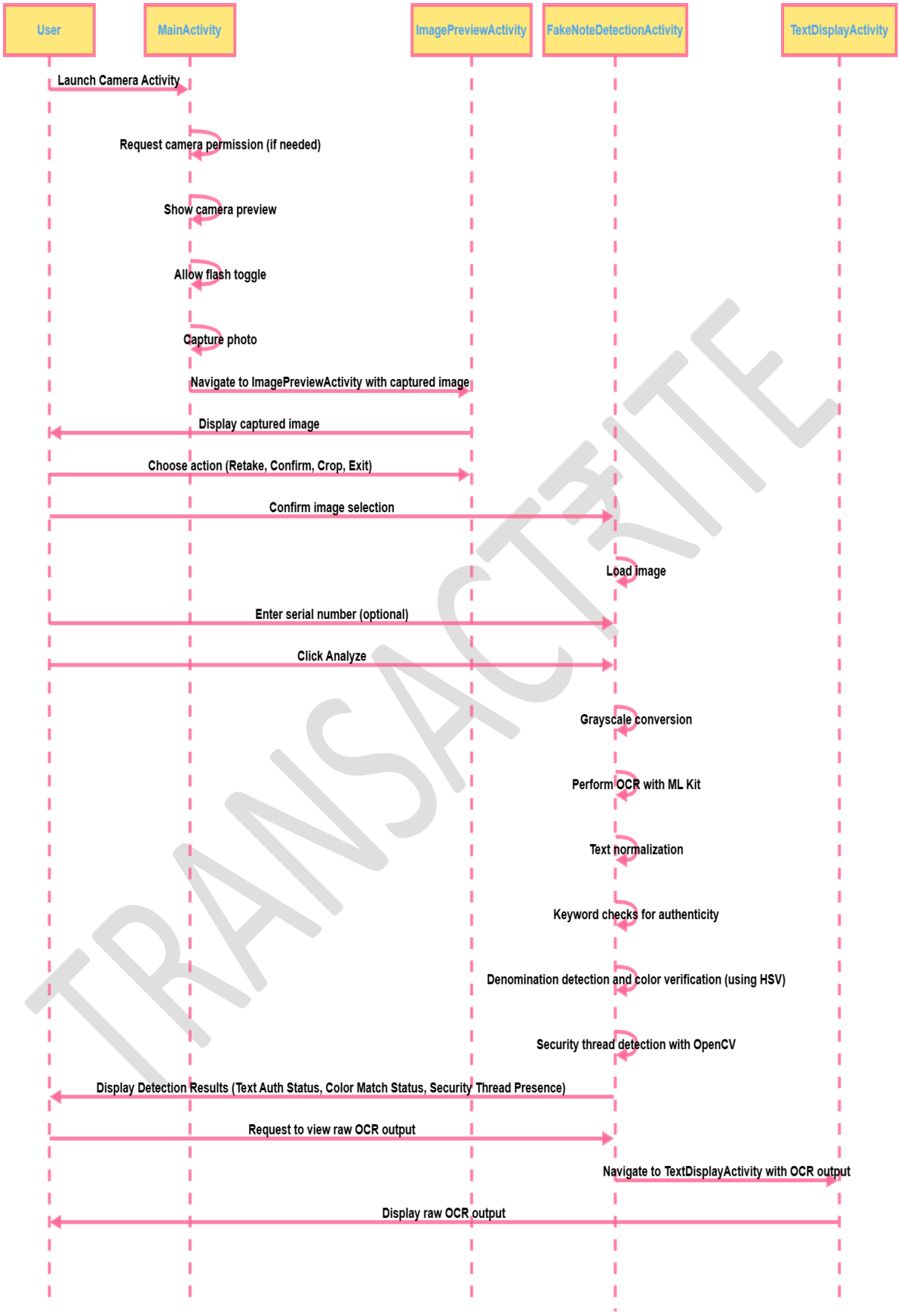
10. Display Analysis Results:

- Update resultTextView with text authentication status.
- Update resultTextView2 with color verification status.
- Update resultTextView3 with security thread detection status.
- Display messages with green color for success and red color for warnings.

11. View Recognized Text (Optional):

- User clicks **View Recognized Text** button.
- Navigates to TextDisplayActivity.
- Displays the full recognized and normalized text extracted from the note.

12. End



FAKE NOTE DETECTION FLOWCHART

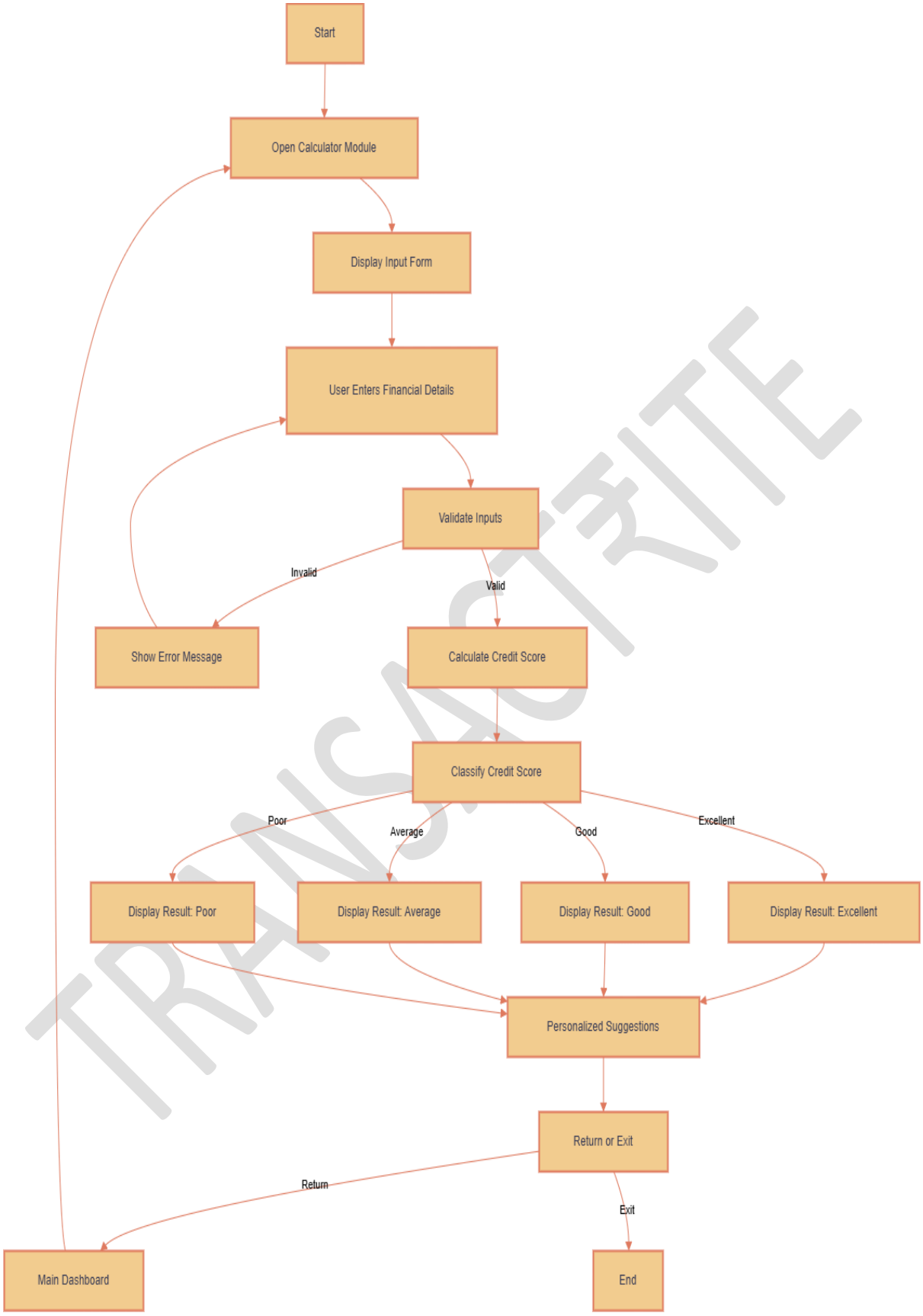
4. Credit Score Calculator

- **Description:** The Credit Score Calculator allows users to input their financial data via a form, calculates their credit score, provides credit improvement tips, and visualizes credit score factors using a pie chart. The user interface is defined in XML, and the logic is implemented in Java.

- **Flowchart steps:**

1. **Start**
    - The CreditScoreActivity is launched, and the UI defined in activity_credit_score.xml is displayed within a ScrollView.
2. **UI Initialization**
    - The UI shows:
        - A title "Credit Score Calculator" (titleText - TextView).
        - Input fields for:
            - Credit Limit (creditLimitLayout - TextInputLayout, creditLimit - EditText).
            - Credit Used (creditUsedLayout - TextInputLayout, creditUsed - EditText).
            - Credit Age (creditAgeLayout - TextInputLayout, creditAge - EditText).
            - Loan Accounts (loanAccountsLayout - TextInputLayout, loanAccounts - EditText).
            - Hard Inquiries (hardInquiriesLayout - TextInputLayout, hardInquiries - EditText).
        - A dropdown to select Payment History (paymentHistoryLayout - TextInputLayout, paymentHistory - AutoCompleteTextView).
        - A "Calculate Credit Score" button (calculateButton - Button).
        - A progress indicator (progressBar - ProgressBar).
        - A text view to display the credit score (resultText - TextView).
        - A pie chart (pieChart - PieChart) to visualize credit factors.
        - A text view to display credit improvement tips (resultTextView2 - TextView).
3. **User Input**
    - The user interacts with the UI by entering data into the EditText fields and selecting an option from the AutoCompleteTextView.
4. **Calculation Trigger** : The user clicks the calculateButton.
5. **Input Validation**
    - The validateInputs() method (in CreditScoreActivity.java) is called.
    - It checks if all input fields have valid data.
    - If any input is invalid:

        - An error message is displayed in the resultText TextView.
        - The process goes to **12. End**.
6. **Score Calculation**
    - If all inputs are valid, the calculateScore() method (in CreditScoreActivity.java) is called.
    - It calculates the credit score based on the input data.
    - If credit used is greater than the credit limit:
        - An error message is displayed in the resultText TextView.
        - The process goes to **12. End**.
7. **Tips Generation**
    - The generateTips() method (in CreditScoreActivity.java) creates personalized credit improvement tips based on the user's input.
8. **UI Update (Score)**
    - The calculated credit score is displayed in the resultText TextView.
9. **UI Update (Tips)**
    - The generated tips are displayed in the resultTextView2 TextView.
10. **UI Update (Chart)**
    - The pieChart is updated to visually represent the contribution of each credit factor.
11. **Progress Indication**
    - The progressBar is used to indicate that the calculation is in progress.
12. **End**



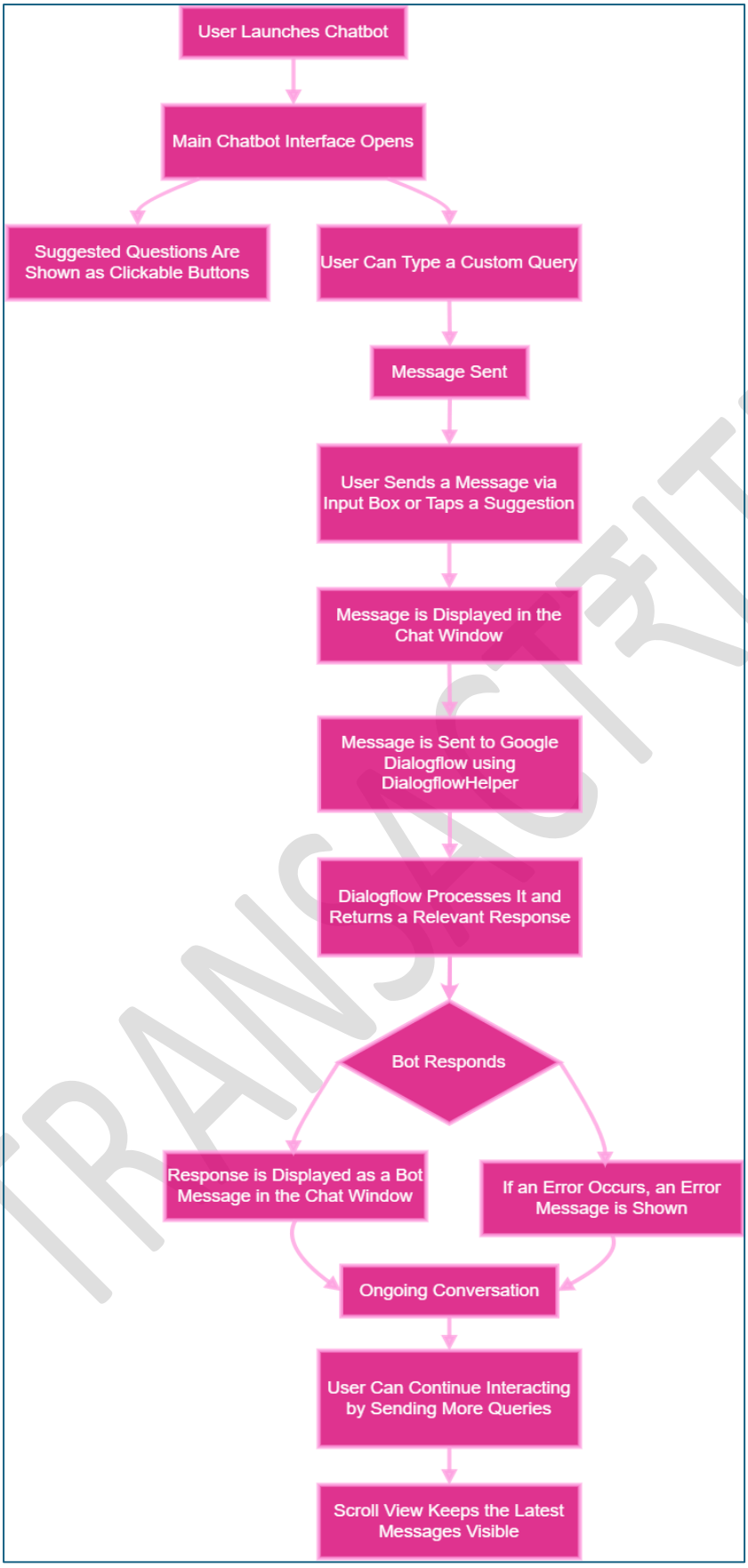
CREDIT SCORE FLOWCHART

5.Chatbot Module Flowchart

- **Description:** This flowchart illustrates the user interaction flow with the chatbot, focusing on message exchange between the user and Dialogflow and the use of suggested questions. The module uses UI from activity_main.xml, chat interface logic from MainActivity.java, and Dialogflow communication via DialogflowHelper.java.
- **Flowchart Steps:**

1. **Start:** MainActivity launches, serving as the chatbot interface.
2. **Initialize:**
    - **Get UI element references:**
        - Retrieves references from activity_main.xml: ImageView (chatbot_icon) for the chatbot icon, LinearLayout (suggestions_layout) for suggested questions, LinearLayout (chat_layout) within ScrollView (scrollView) for chat history, EditText (etUserMessage) for user input, and Button (btnSend) to send messages.
    - **Initialize DialogflowHelper:**
        - Instantiates DialogflowHelper to manage Dialogflow communication, including credential loading and session setup.
    - **Set up suggested question buttons:**
        - Dynamically creates and adds Button objects to suggestions_layout, providing predefined conversation starters.
3. **User Action:** User interacts by:
    - Typing a message in etUserMessage,
    - OR, clicking a suggested question button.
4. **Calculate Trigger:** Message sending is triggered by:
    - Clicking btnSend (after typing),
    - OR, clicking a suggestion button.
5. **Get Input:**
    - Retrieves user's message from etUserMessage,
    - OR, populates etUserMessage with the clicked suggestion's text.
6. **Display User Message:**
    - addMessage() in MainActivity displays the user's message in chat_layout (creating and formatting a TextView).
7. **Send to Dialogflow:**
    - sendMessage() in DialogflowHelper sends the user's message to Dialogflow.

8. **Receive Response:**
    - Dialogflow processes the message and generates a response.
9. **Display Bot Response:**
    - addMessage() (via ResponseCallback) in MainActivity displays the bot's response in chat_layout.
10. **Display Suggestions:**
    - addSuggestion() in MainActivity displays suggested question buttons (initially and potentially during the conversation); clicking a button populates the input and sends it.
11. **End:** Conversation continues until the user exits.

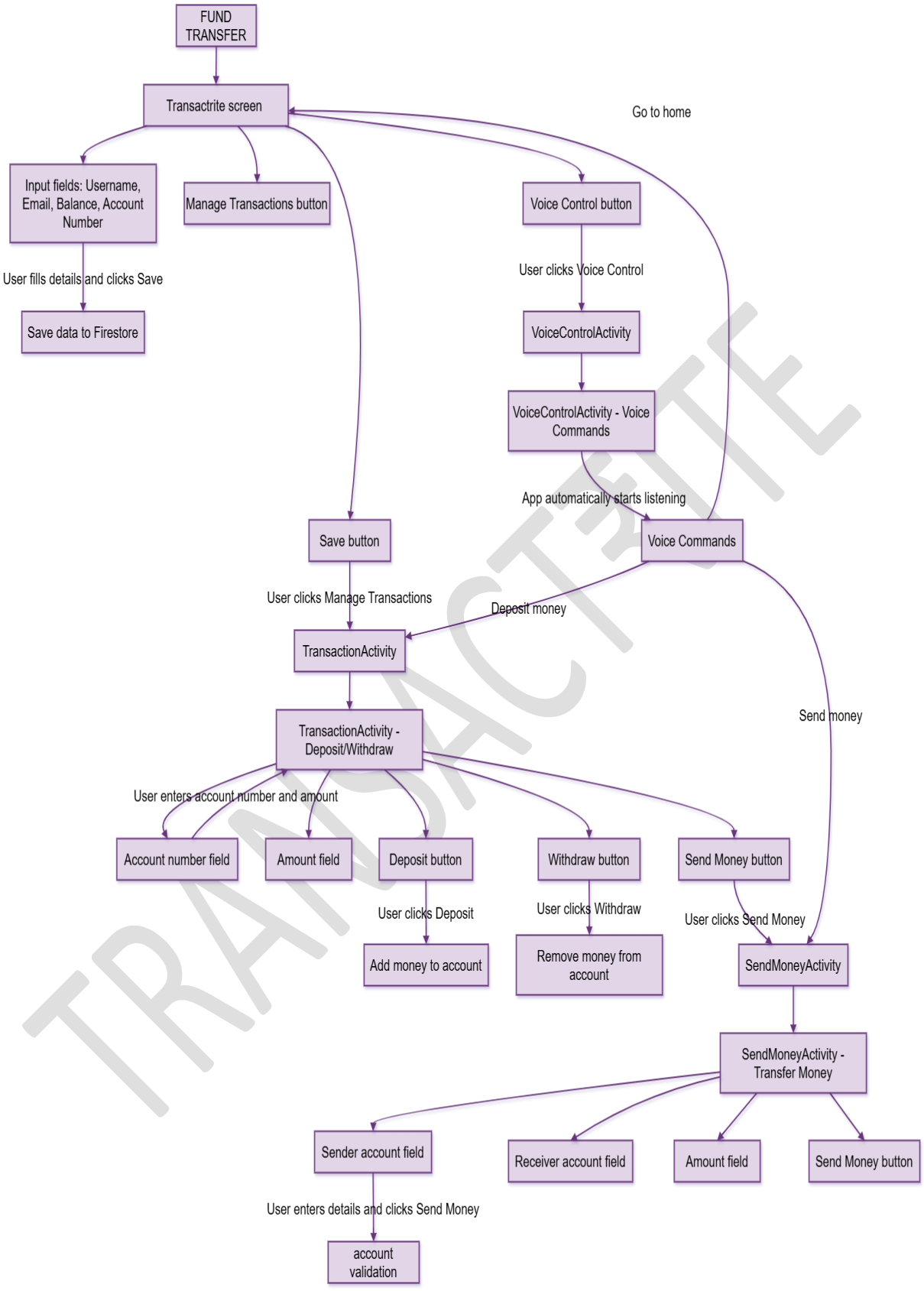


CHATBOT FLOWCHART

6. Fund Transfer Module Flowchart

- **Description:** This flowchart illustrates the process of initiating a fund transfer, validating input, checking account existence, updating balances, and recording the transaction using Firebase Firestore.
- **Flowchart Steps:**
    1. **Start**
    2. **Initialize:**
        - Initialize Firebase Firestore.
        - Set up references to UI elements:
            - EditText (etAmount, etSenderAccount, etReceiverAccount)
            - Button (btnSendMoney, btnTransactionHistory)
        - Retrieve or prepare any necessary data.
    3. **User Action:**
        - User enters the sender's account number, receiver's account number, and the amount to transfer in the respective input fields.
        - User clicks the "Send Money" button (btnSendMoney).
    4. **Validate Input:**
        - Check if all input fields (sender's account, receiver's account, amount) are filled.
        - If any field is empty, display an error message and stop the process.
        - Attempt to convert the entered amount to a double.
        - If the conversion fails (e.g., invalid format), display an "Invalid amount" error and stop.
        - Check if the amount is greater than zero.
        - If the amount is not positive, display an error message and stop.
    5. **Check Receiver Existence:**
        - Query the "users" collection in Firestore to check if the receiver's account number exists.
        - If the receiver's account does not exist, display an error message ("Receiver does not exist") and stop.
    6. **Perform Transaction (Firestore Transaction):**
        - Initiate a Firestore transaction to ensure atomicity.
        - Inside the transaction:
            - Retrieve the current balance of the sender's account.
            - Retrieve the current balance of the receiver's account.
            - Check if the sender has sufficient funds (sender's balance >= transfer amount).
            - If the sender has insufficient funds, abort the transaction, display an error ("Insufficient funds"), and stop.
            - Calculate the new sender balance (sender's balance - transfer amount).

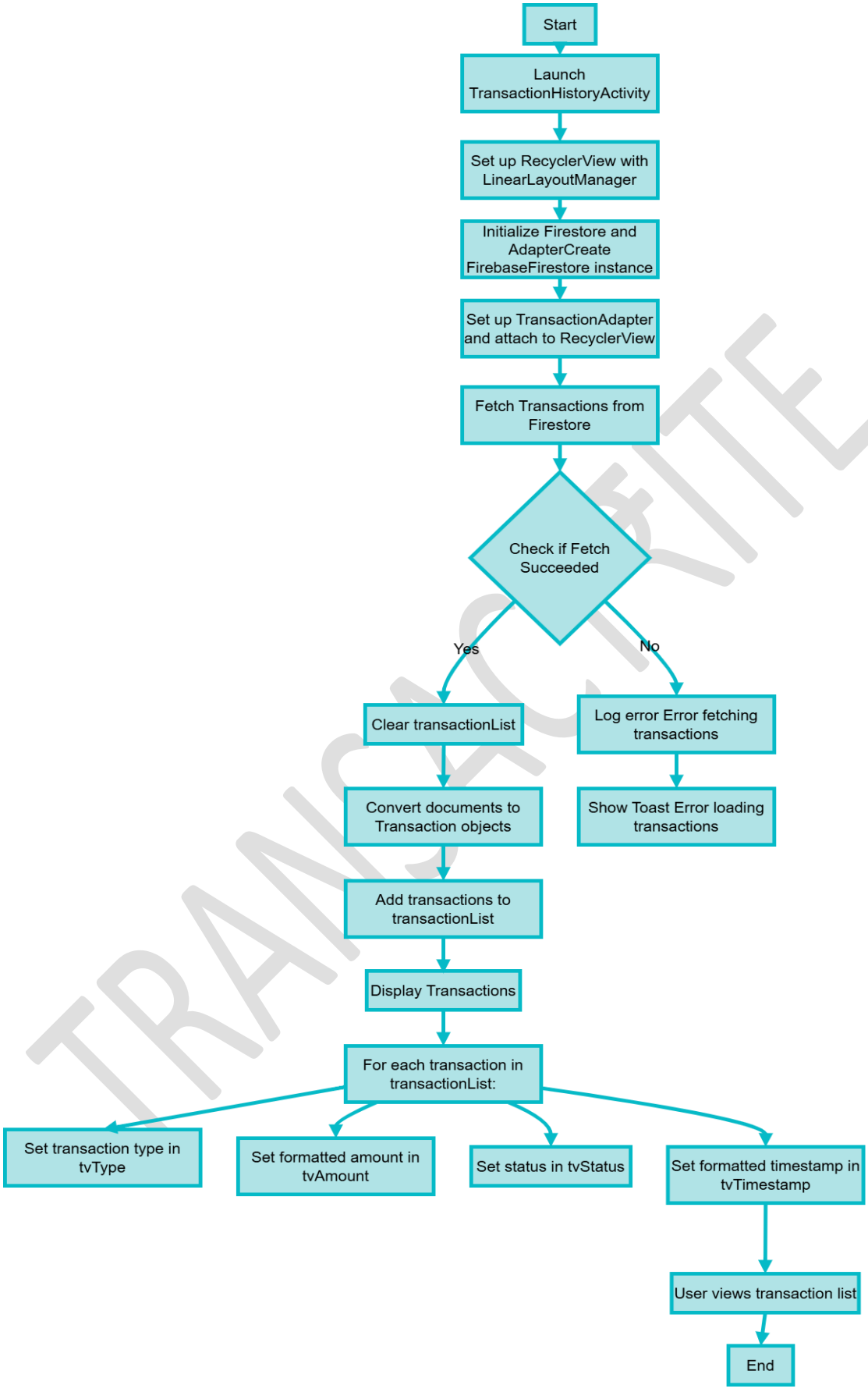
        - Calculate the new receiver balance (receiver's balance + transfer amount).
        - Update the sender's balance in the "users" collection with the new balance.
        - Update the receiver's balance in the "users" collection with the new balance.
    - If the Firestore transaction is successful, proceed to the next step.
    - If the transaction fails, display an error message ("Transaction failed") and stop.
7. **Record Transaction:**
    - Generate a unique transaction ID.
    - Create a new document in the "transactions" collection in Firestore.
    - Store the following transaction details:
        - transactionId (the generated ID)
        - sender (sender's account number)
        - receiver (receiver's account number)
        - amount (transfer amount)
        - type ("Transfer")
        - timestamp (current timestamp)
8. **Display Result:**
    - If all steps are successful, display a success message ("Transaction successful").
    - If any error occurred during the process, display the corresponding error message to the user.
9. **Transaction History (Optional):**
    - User clicks the "Transaction History" button (btnTransactionHistory).
    - Navigates to the Transaction History Activity.
    - Displays the transaction history.
10. **End**



FUND TRANSFER FLOWCHART

7. Transaction History Module Flowchart

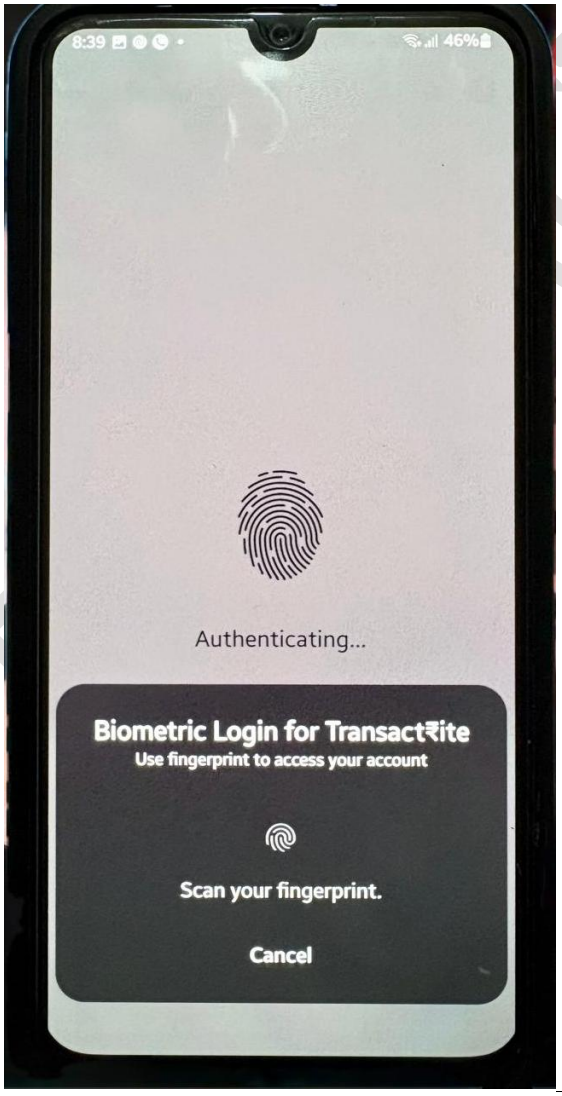
- **Description:** This flowchart illustrates the process of retrieving and displaying transaction history from Firebase Firestore in a RecyclerView.
- **Flowchart Steps:**
    1. **Start**
    2. **Initialize:**
        - Initialize Firebase Firestore.
        - Set up references to UI elements:
            - RecyclerView (recyclerViewTransactions)
        - Create an empty list to hold Transaction objects (transactionList).
        - Create a TransactionAdapter instance (adapter) and associate it with the transactionList.
        - Set the layout manager of the recyclerViewTransactions to LinearLayoutManager.
        - Set the adapter to the recyclerViewTransactions.
    3. **Fetch Transactions:**
        - Query the "transactions" collection in Firestore to retrieve all transaction documents.
        - Set up listeners for success and failure of the Firestore query.
    4. **On Success (Firestore Query):**
        - Clear the transactionList to remove any previous data.
        - Iterate through each document (QueryDocumentSnapshot) in the result:
            - Convert the document data into a Transaction object.
            - Add the Transaction object to the transactionList.
            - Notify the adapter that a new item has been inserted at the end of the list (adapter.notifyItemInserted()). This is more efficient than notifyDataSetChanged().
    5. **On Failure (Firestore Query):**
        - Log the error message.
        - Display a Toast message to the user indicating that there was an error loading transactions.
    6. **Display Transactions (TransactionAdapter):**
        - The TransactionAdapter is responsible for taking the transactionList and displaying each Transaction object in a row of the RecyclerView.
        - For each transaction:
            - Set the text of the tvType TextView to the transaction's type.
            - Set the text of the tvAmount TextView to the transaction's amount, formatted with a currency label.
            - Set the text of the tvStatus TextView to the transaction's status.
            - Set the text of the tvTimestamp TextView to the transaction's timestamp, formatted as "dd/MM/yyyy HH:mm".
    7. **End**

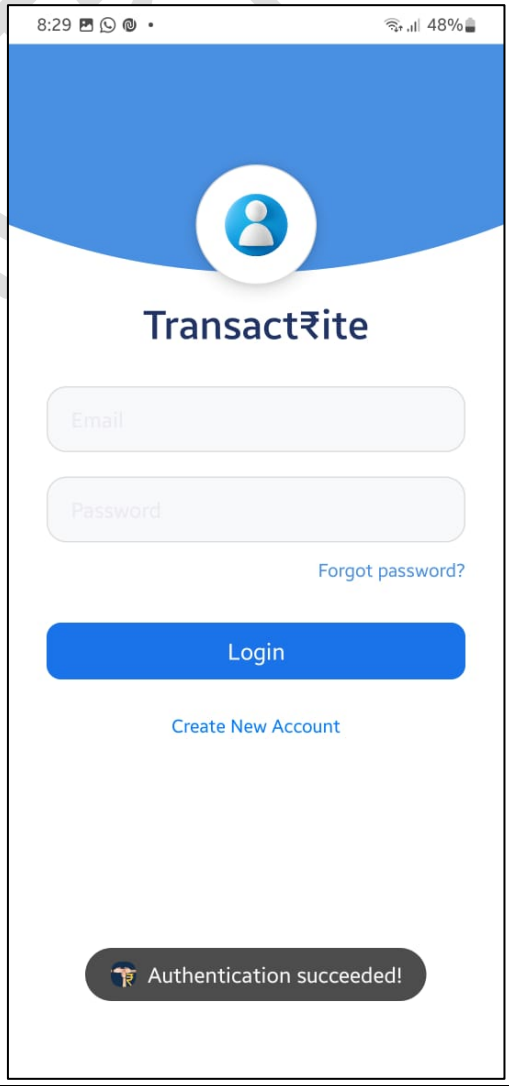


TRANSACTION MODULE FLOWCHART

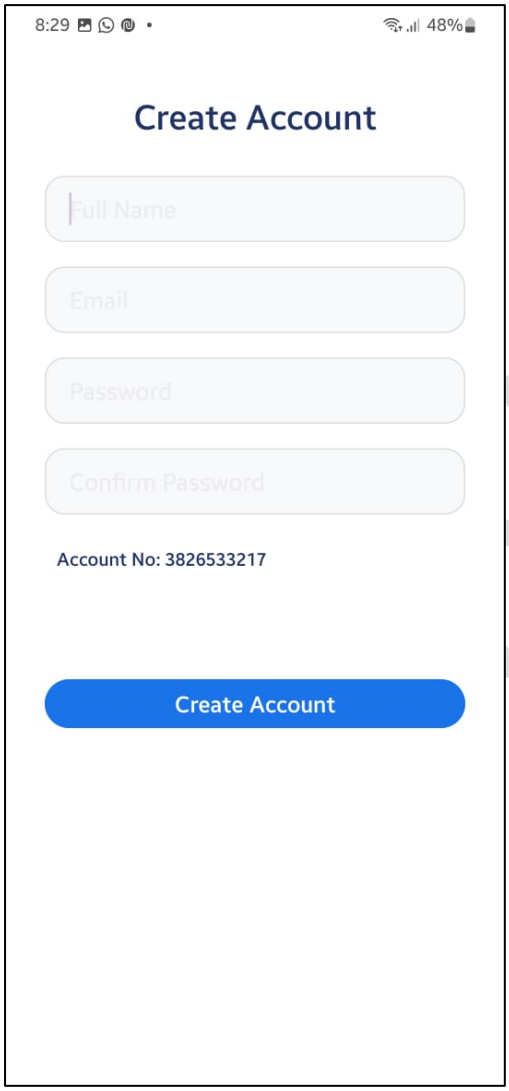
9.COMPLETE SNAPSHOTS

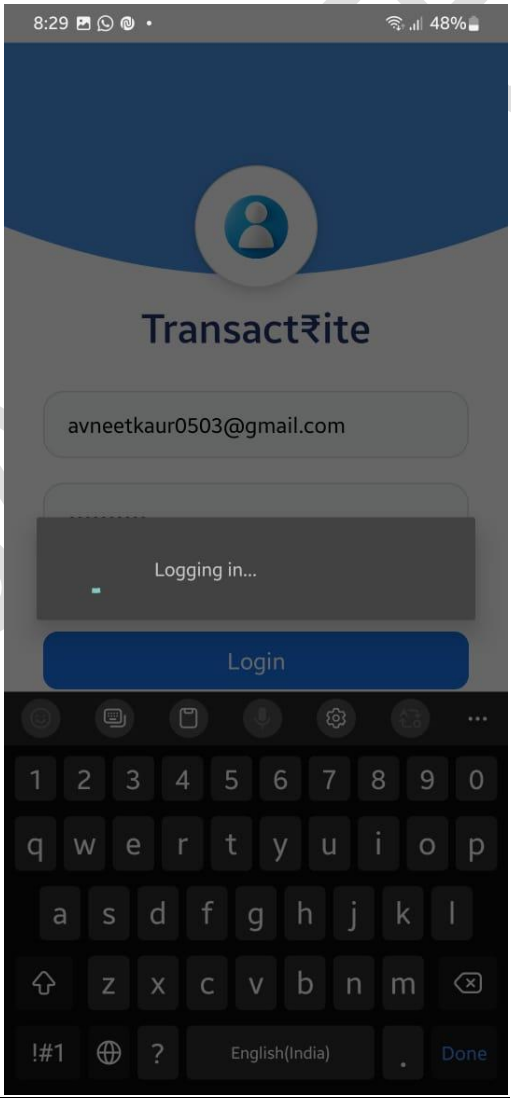
Biometric and login screen:-



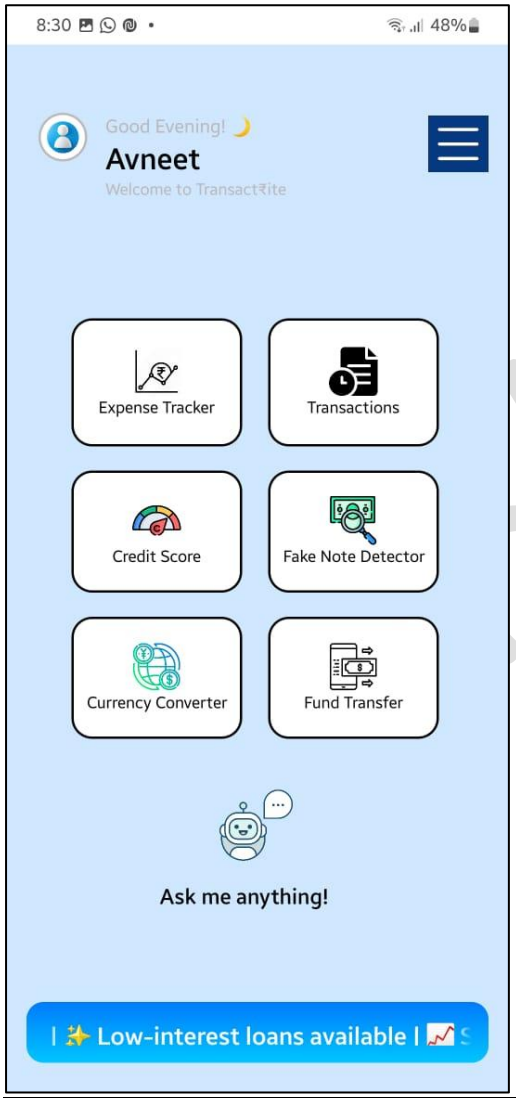


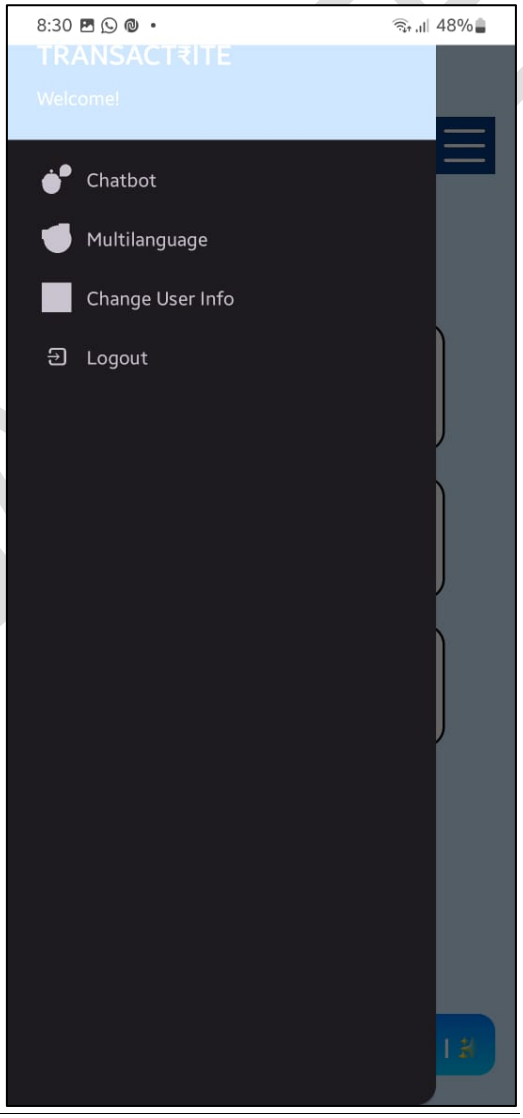
Create account screen and login



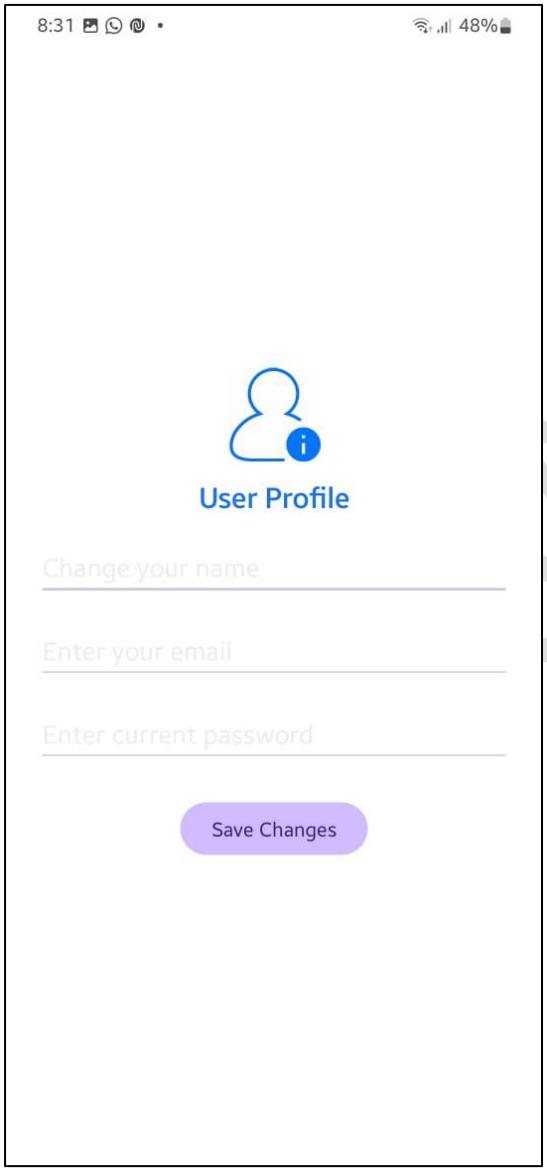


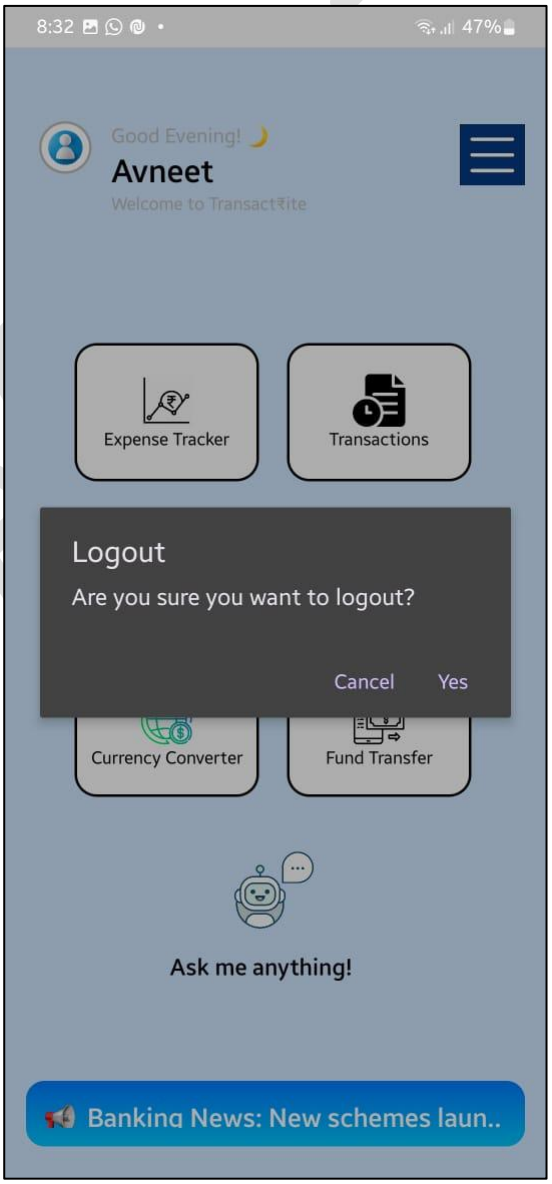
Dashboard and side navigation panel



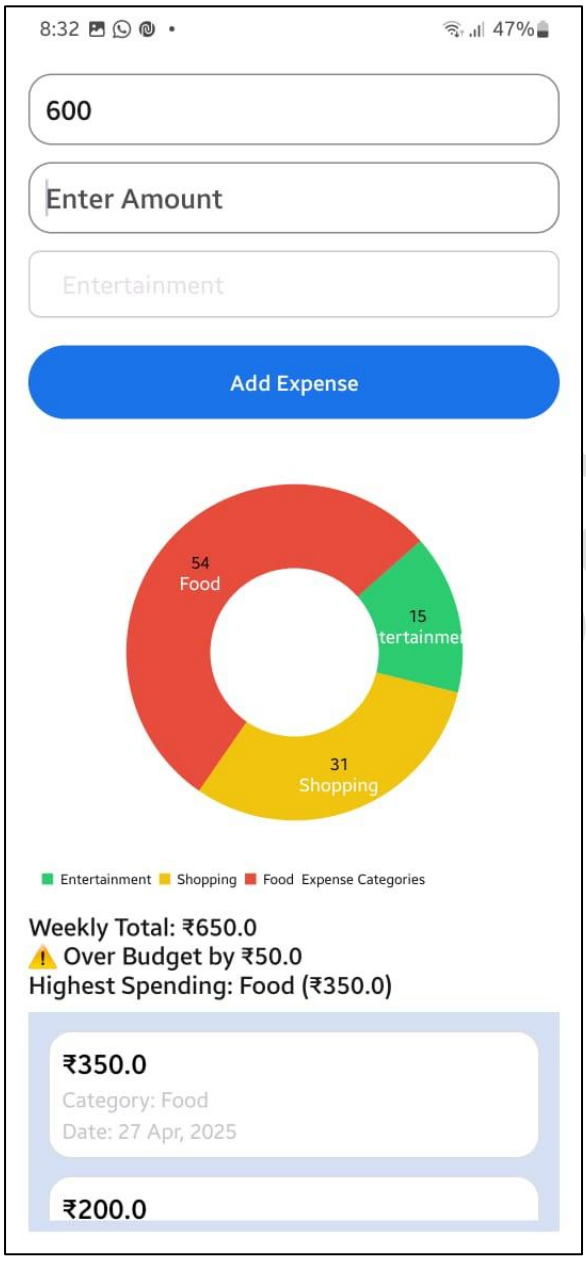


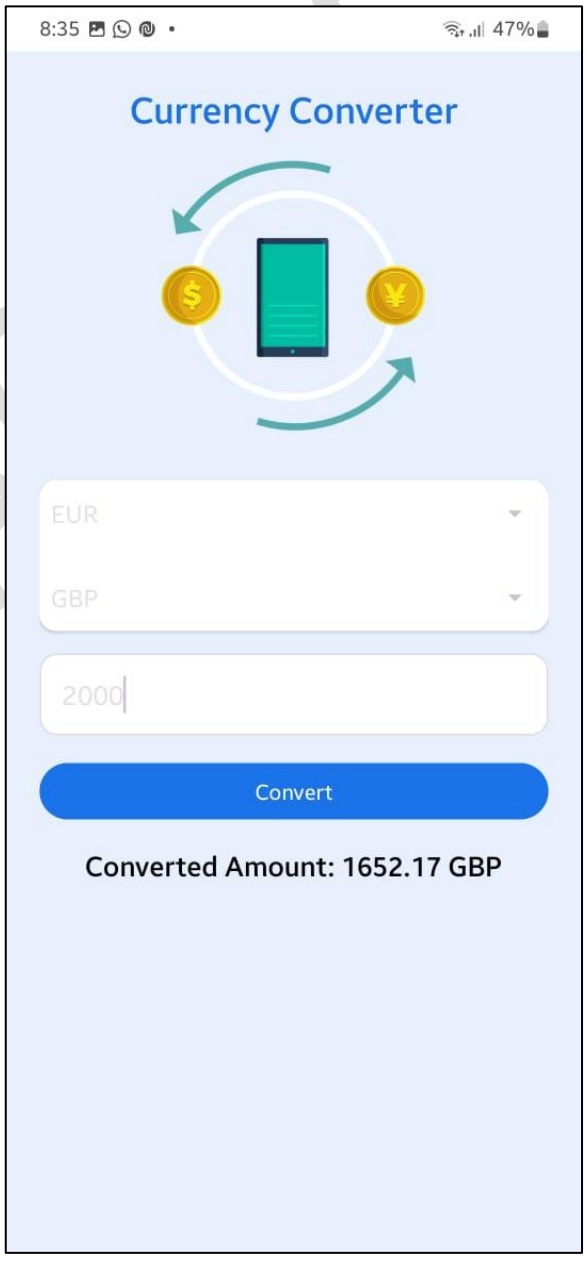
Change user info and logout



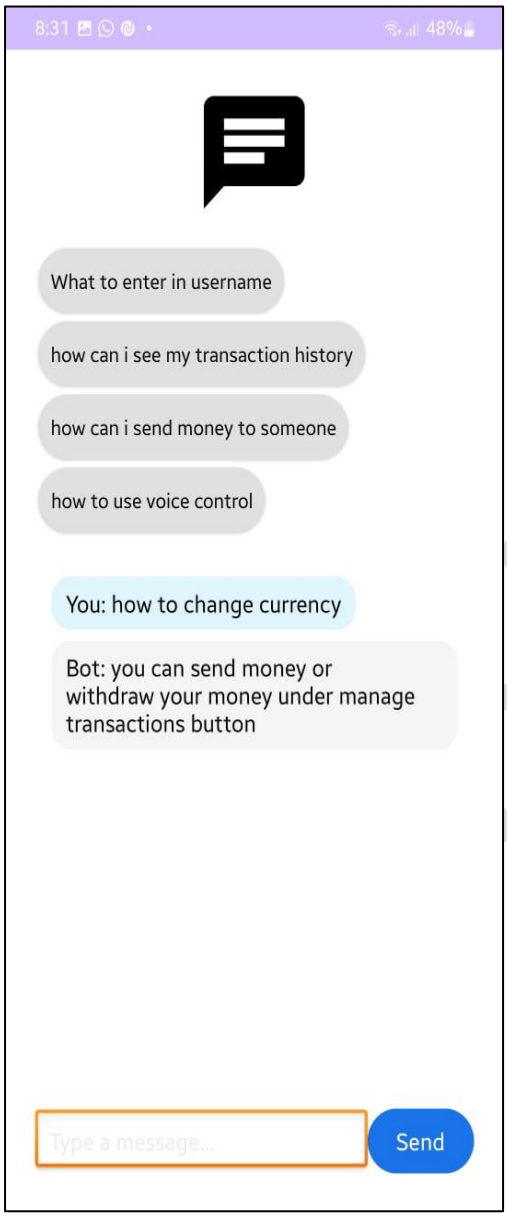


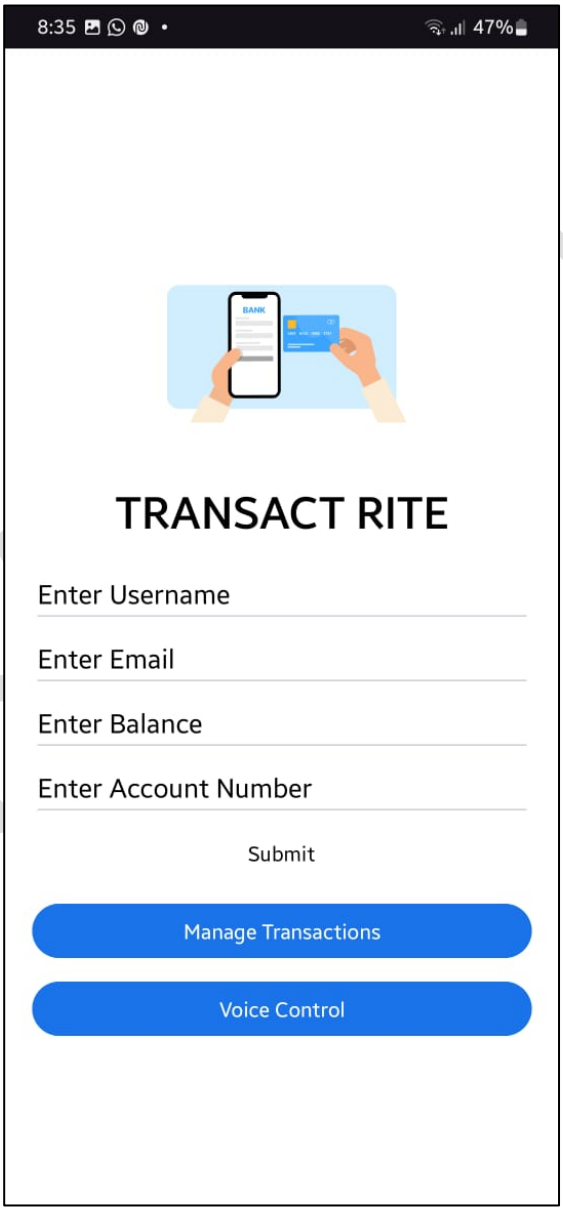
Expense tracker and currency converter



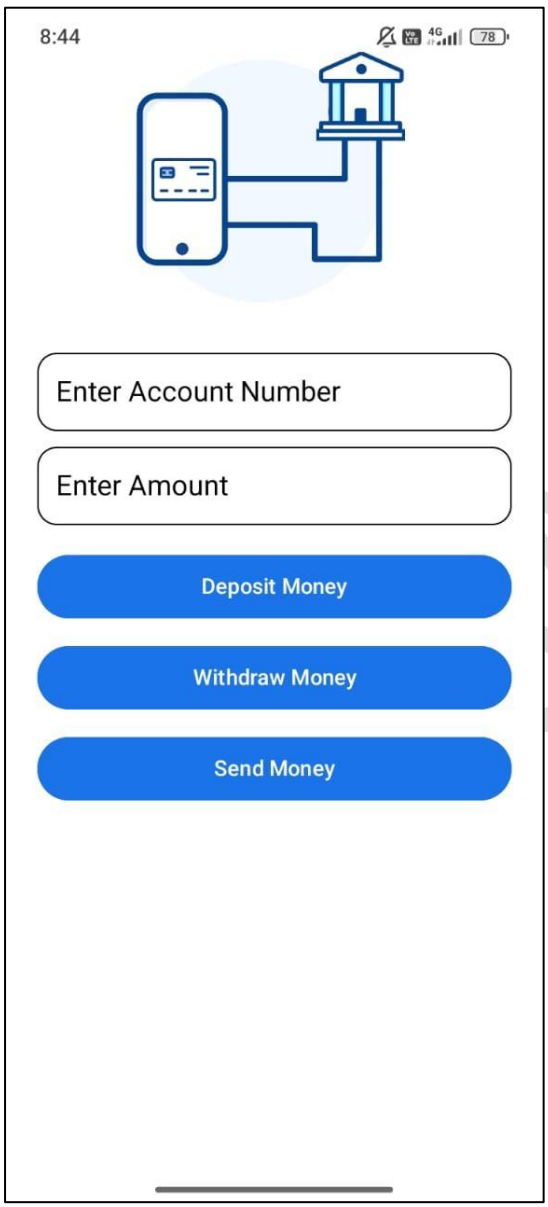


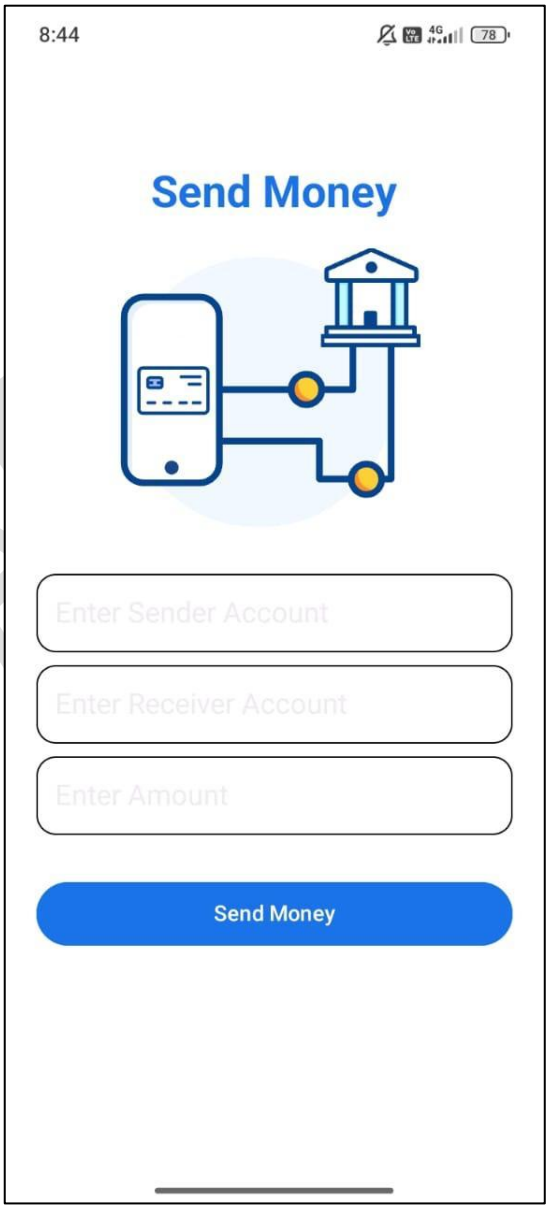
Chatbot and Fund Transfer



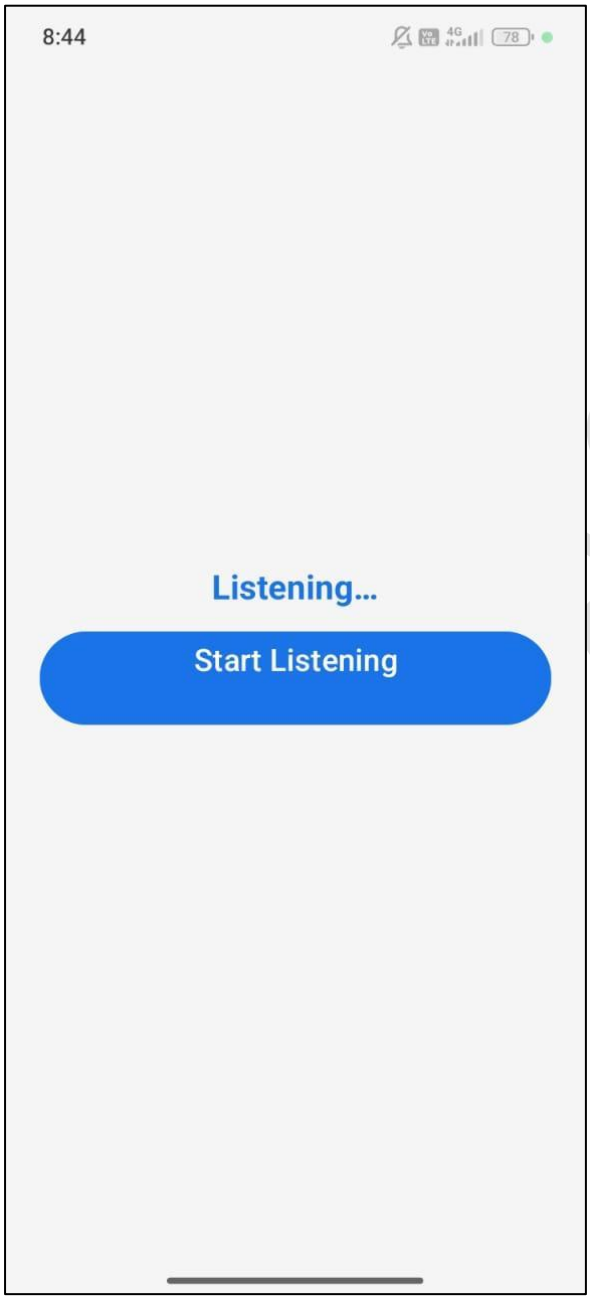


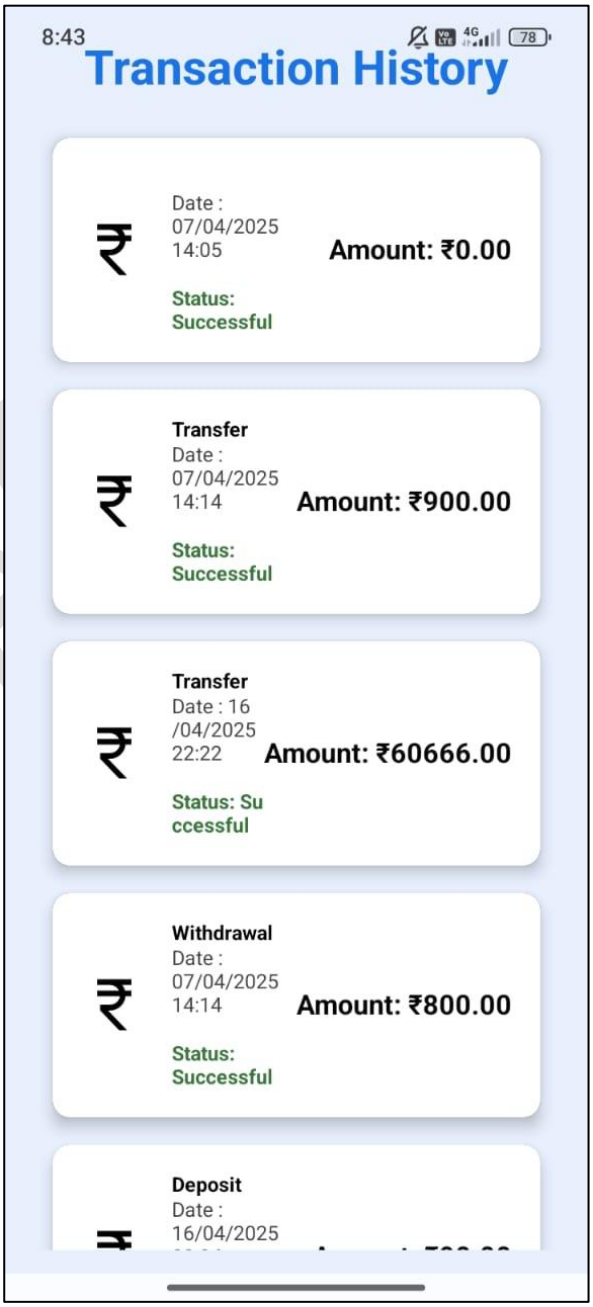
Deposit And Withdrawal Screen And Send Money Screen



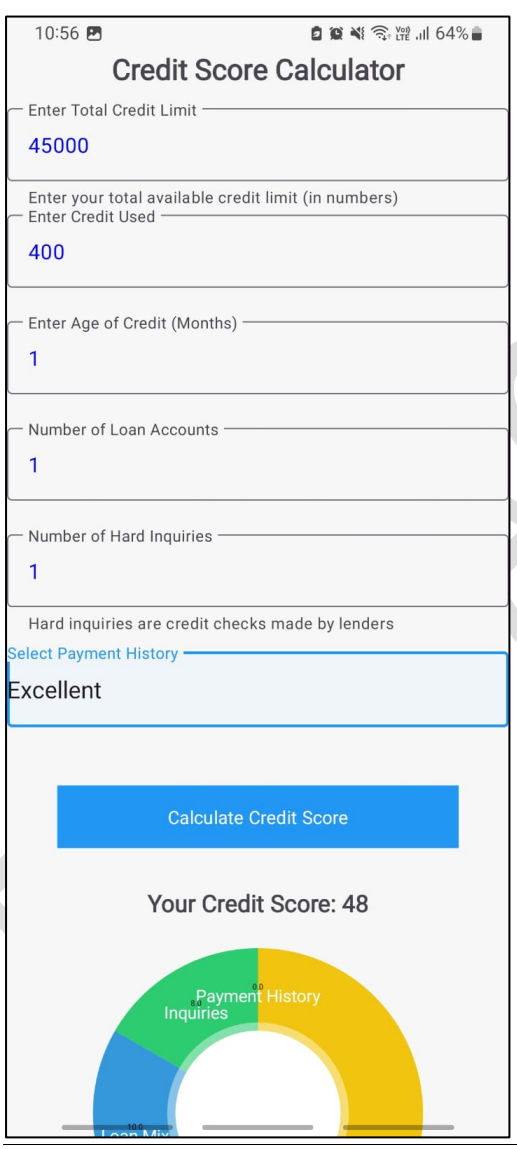


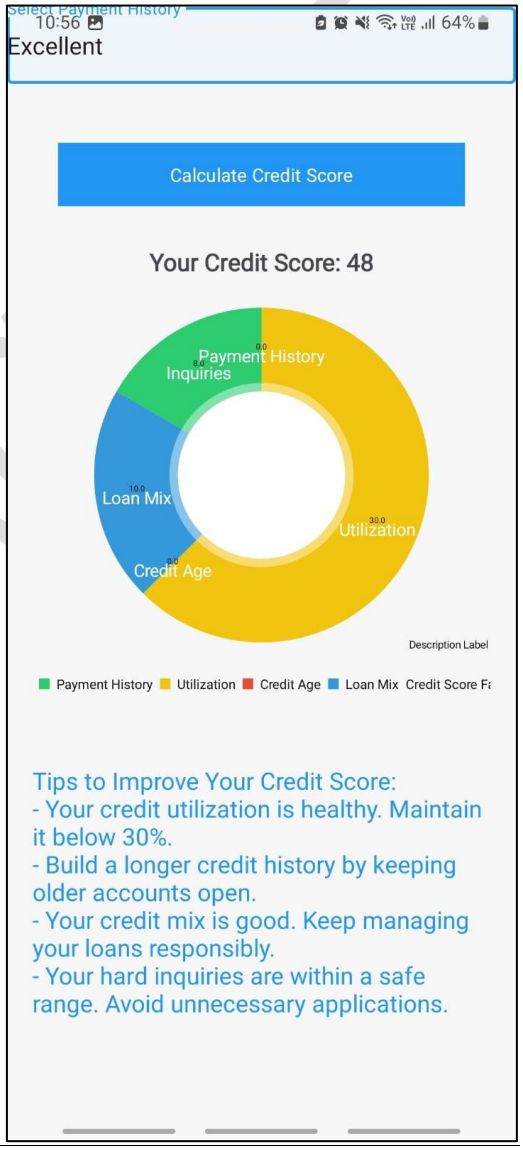
Voice Control And Transactions



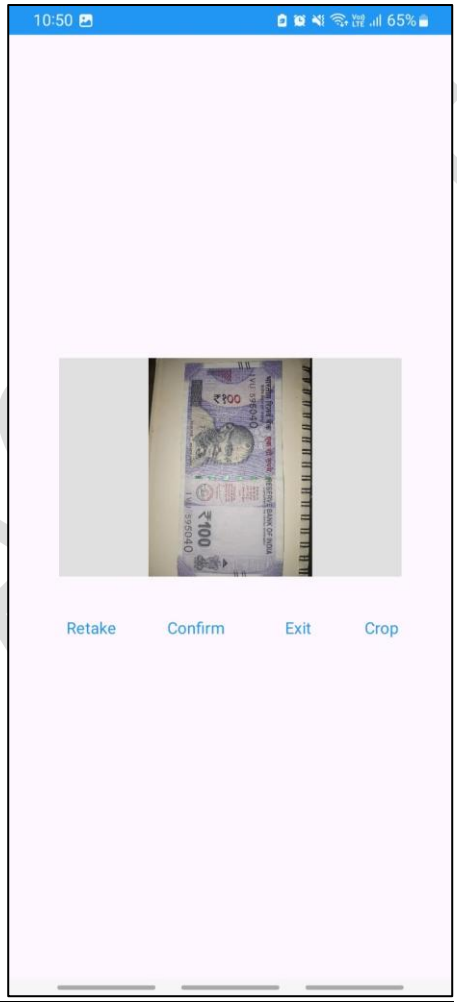


Credit Score Calculator (before and after)

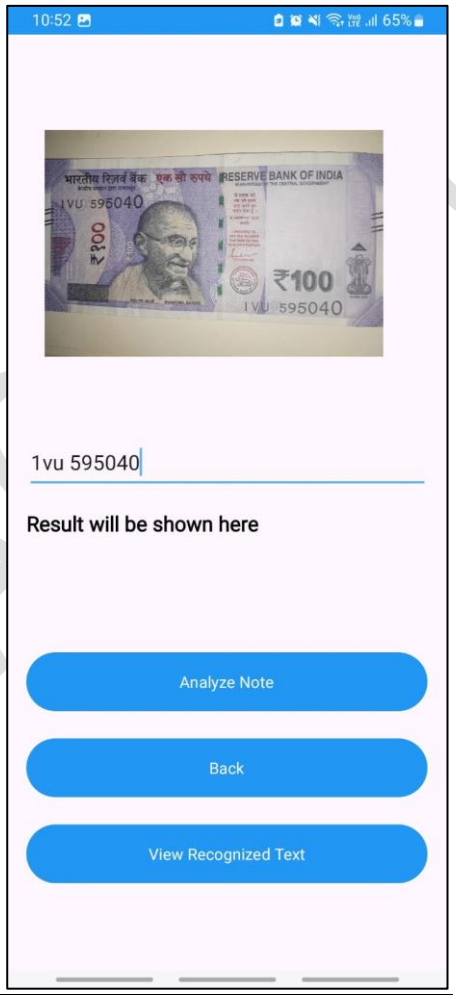




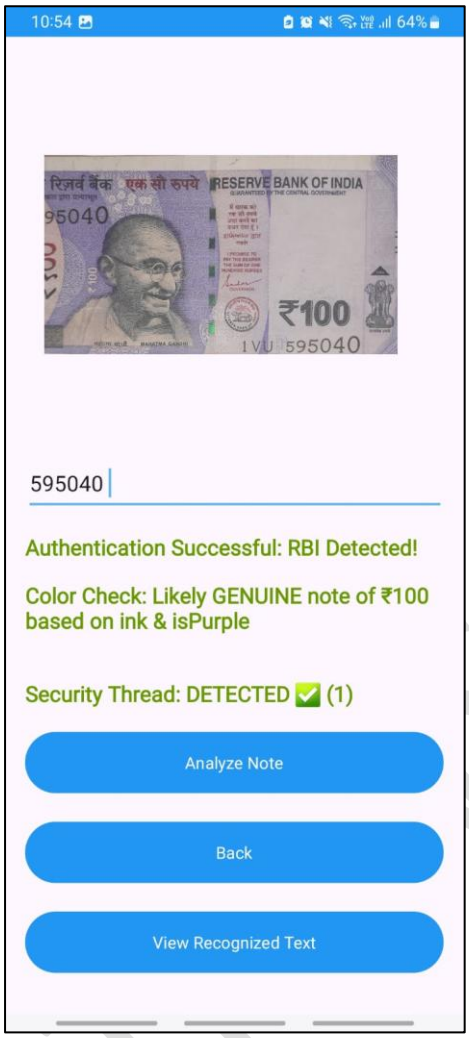
Fake Note Detector Capture Screen And Preview Screen

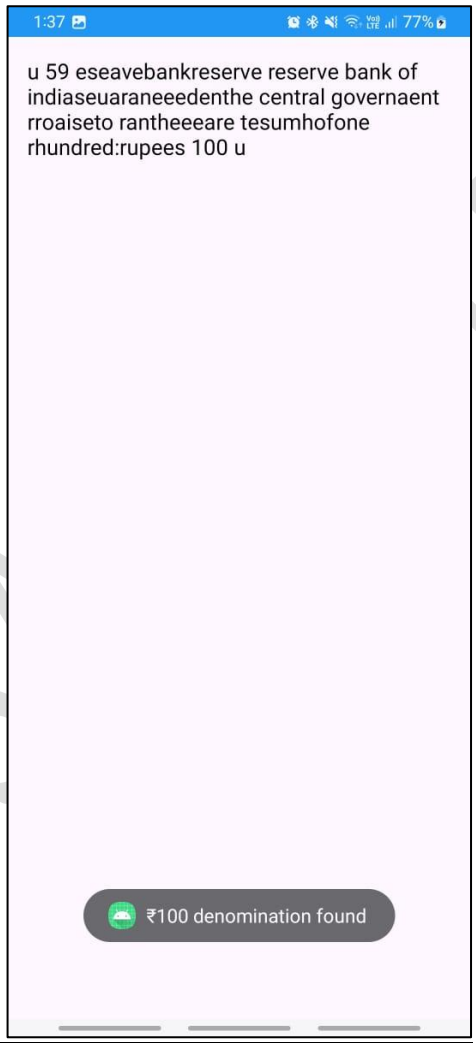


Edit Image Screen And Processing Screen



Result Screen With Recognized Text (Optional)





10) FUTURE SCOPE OF THE PROJECT

The **Transact₹ite Banking App** is designed with scalability, innovation, and user-centric features in mind. To ensure continuous growth, competitiveness, and user satisfaction, the following future enhancements and developments are planned:

1. **Integration with Real-Time Payment Gateways**
    - Incorporate support for UPI, NEFT, IMPS, Razorpay, PayPal, and Stripe to enable direct bank-to-bank transactions and multi-currency support.
    - Implement instant fund transfers across different banks and financial institutions.
2. **AI-powered Financial Services**
    - Integrate AI-based financial recommendation systems to suggest saving plans, loan offers, and investment strategies based on user behavior and transaction history.
    - Develop AI-driven fraud detection mechanisms for early identification of suspicious activities.
3. **Advanced Fraud and Fake Note Detection**
    - Expand the Fake Note Detection module to support multiple international currencies.
    - Integrate cloud-based fraud detection APIs and enhance model accuracy with advanced deep learning (AI/ML) models.
4. **Enhanced Credit Score Analysis**
    - Enable PDF exports of personalized credit reports.
    - Integrate third-party credit score APIs (like CIBIL, Experian) for real-time and official credit score retrieval.
    - Add graphical visualizations and historical analysis of the user's credit score trends.
5. **Improved User Experience (UI/UX)**
    - Introduce dark mode, theme customization, and dynamic UI elements.
    - Implement QR Code-based payments for faster fund transfers.

- Add Voice Assistant features for complete voice-controlled banking navigation.

6. **Cloud Integration and Scalability**
    - Migrate transaction history and other critical data to scalable cloud databases (such as Google Cloud Firestore) for faster access and backup.
    - Implement real-time push notifications for transaction updates, security alerts, and offers.
7. **Security and Privacy Enhancements**
    - Introduce Multi-Factor Authentication (MFA) as a mandatory feature for sensitive transactions.
    - Integrate biometric authentication layers such as Face Recognition in addition to Fingerprint Authentication.
8. **Analytics and User Insights**
    - Develop a personalized spending analytics dashboard using charts and graphs to help users manage their finances better.
    - Offer monthly expense summaries, budget comparisons, and future savings predictions.
9. **International Expansion**
    - Customize modules to support international banking standards and financial regulations.
    - Add multi-language support and currency converters for global users.
10. **Eco-Friendly Initiatives**
    - Implement digital document generation (statements, receipts) to reduce paper usage.
    - Promote Green Banking initiatives through app notifications and campaigns.

11) CONCLUSION

The **Transact₹ite Banking Application** has been designed and developed as a modern, user-friendly, and secure mobile banking platform that addresses the essential needs of today's digital banking users. With core features such as secure login authentication (using Firebase and biometric authentication), a responsive dashboard, seamless fund transfers, real-time transaction history, fake note detection using image processing, and an interactive credit score calculator, the app delivers a comprehensive and innovative banking experience.

Throughout the development process, special attention was given to ensuring security, performance, usability, and scalability. By integrating cloud technologies like Firebase and leveraging AI-driven modules such as Fake Note Detection and Credit Score Calculation, Transact₹ite demonstrates the potential for smart, future-ready banking solutions.

The app not only fulfills current user requirements but also lays a strong foundation for future enhancements such as UPI and international payment integration, AI-powered financial recommendations, real-time fraud detection, and improved credit score management.

In conclusion, **Transact₹ite** successfully meets the project's objectives by providing an intuitive, secure, and dynamic banking platform. It represents a significant step toward digital transformation in the banking sector and holds great potential for further development to meet evolving user expectations and technological advancements.